
title: "JavaScript DOM: O Guia Definitivo"

subtitle: "Do Básico ao Avançado — Todos os Eventos, Propriedades e Métodos com Exemplos Práticos"

author: "Manus AI"

date: "2026"

JavaScript DOM: O Guia Definitivo

Do Básico ao Avançado — Todos os Eventos, Propriedades e Métodos com Exemplos Práticos

Autor: Manus AI

Edição: 2026

Prefácio

Esta apostila foi criada para ser o recurso mais completo em língua portuguesa sobre o Document Object Model (DOM) do JavaScript. Seja você um iniciante que está dando os primeiros passos no desenvolvimento web, ou um desenvolvedor experiente que deseja consolidar e aprofundar seus conhecimentos, este material foi estruturado para atender a todos os níveis.

O DOM é o coração do desenvolvimento Frontend. Antes de aprender qualquer framework moderno como React, Vue ou Angular, é fundamental dominar o JavaScript puro e sua interação com o DOM. Frameworks são abstrações construídas sobre o DOM — entendê-lo profundamente torna você um desenvolvedor muito mais eficaz e capaz de depurar problemas que as abstrações escondem.

Ao longo dos 20 capítulos desta apostila, você encontrará explicações teóricas detalhadas, centenas de exemplos de código comentados, tabelas de referência rápida e projetos práticos completos. Cada conceito é apresentado de forma progressiva, construindo sobre o conhecimento anterior.

Sumário

Capítulo	Título	Página
1	Introdução ao DOM	5
2	Seleção de Elementos	18
3	Navegação na Árvore do DOM	32
4	Manipulação de Conteúdo	44
5	Atributos, Propriedades e Dataset	56
6	Classes e Estilos CSS	68
7	Criação e Inserção de Elementos	82
8	Remoção e Substituição de Elementos	95
9	Introdução a Eventos	104
10	Eventos de Mouse e Pointer	118
11	Eventos de Teclado	130
12	Eventos de Formulário	140
13	Eventos de Janela e Documento	152
14	Eventos Avançados e Customizados	162
15	O BOM e APIs Web	170
16	Web Storage e Cookies	180
17	Drag and Drop API	188
18	MutationObserver e IntersectionObserver	196
19	Canvas e SVG no DOM	206
20	Projetos Práticos Completos	216
Apêndice	Tabelas de Referência Rápida	240

Capítulo 1: Introdução ao DOM

1.1 O Que é o DOM?

O **Document Object Model (DOM)** é uma interface de programação para documentos web. Ele representa a estrutura de um documento HTML ou XML como uma árvore de objetos, onde cada nó da árvore representa uma parte do documento. Por meio do DOM, programas como o JavaScript podem acessar e manipular o conteúdo, a estrutura e o estilo de um documento de forma dinâmica.

É importante compreender que o DOM **não é** uma linguagem de programação, mas sim uma API (Application Programming Interface) que define como os programas podem acessar e modificar documentos. O DOM é independente de linguagem — embora o JavaScript seja a linguagem mais comumente usada para interagir com ele em navegadores, o DOM pode ser acessado por qualquer linguagem de programação.

"O Document Object Model é uma interface de plataforma e linguagem neutra que permite que programas e scripts acessem e atualizem dinamicamente o conteúdo, a estrutura e o estilo de um documento." — W3C

A especificação do DOM é mantida pelo **World Wide Web Consortium (W3C)** e pelo **WHATWG (Web Hypertext Application Technology Working Group)**. A versão atual e viva da especificação é mantida pelo WHATWG em <https://dom.spec.whatwg.org/>.

1.2 A Árvore do DOM: Nós e Hierarquia

Quando um navegador carrega uma página HTML, ele analisa (parse) o código HTML e constrói uma representação interna em memória chamada de **árvore do DOM**. Esta árvore é composta por **nós** (nodes), organizados hierarquicamente.

Considere o seguinte HTML simples:

```
<!DOCTYPE html>
<html lang="pt-BR">
  <head>
    <title>Minha Página</title>
  </head>
  <body>
    <h1 id="titulo">Olá, Mundo!</h1>
    <p class="intro">Este é um parágrafo.</p>
  </body>
</html>
```

A árvore do DOM correspondente seria estruturada da seguinte forma:

```
Document
├─ html (Element)
│   ├─ head (Element)
│   │   └─ title (Element)
│   │       └─ "Minha Página" (Text)
│   └─ body (Element)
│       ├─ h1#titulo (Element)
│       │   └─ "Olá, Mundo!" (Text)
│       └─ p.intro (Element)
│           └─ "Este é um parágrafo." (Text)
```

Cada item nessa representação é um **nó**. O nó raiz é sempre o objeto `Document`.

1.3 Tipos de Nós

O DOM define vários tipos de nós, cada um com sua própria interface e comportamento. Os mais importantes são listados na tabela a seguir:

Constante	Valor Numérico	Descrição
<code>Node.ELEMENT_NODE</code>	1	Representa um elemento HTML como <code><p></code> , <code><div></code> , <code><a></code>
<code>Node.TEXT_NODE</code>	3	Representa o conteúdo de texto dentro de um elemento
<code>Node.COMMENT_NODE</code>	8	Representa um comentário HTML <code><!-- ... --></code>
<code>Node.DOCUMENT_NODE</code>	9	Representa o documento inteiro (o objeto <code>document</code>)
<code>Node.DOCUMENT_TYPE_NODE</code>	10	Representa a declaração <code><!DOCTYPE html></code>
<code>Node.DOCUMENT_FRAGMENT_NODE</code>	11	Representa um fragmento de documento (usado para performance)

Você pode verificar o tipo de um nó usando a propriedade `nodeType`:

```
const h1 = document.querySelector("h1");
console.log(h1.nodeType);      // 1 (ELEMENT_NODE)
console.log(h1.nodeName);     // "H1"
console.log(h1.nodeValue);    // null (elementos não têm nodeValue)

const textoDoH1 = h1.firstChild;
console.log(textoDoH1.nodeType); // 3 (TEXT_NODE)
console.log(textoDoH1.nodeName); // "#text"
console.log(textoDoH1.nodeValue); // "Olá, Mundo!"
```

1.4 O Objeto `document`

O objeto `document` é o ponto de entrada para toda a árvore do DOM. Ele representa a página inteira e fornece métodos para criar, encontrar e manipular elementos.


```
// Propriedades informativas do documento
console.log(document.title);      // Título da página
console.log(document.URL);        // URL completa da página
console.log(document.domain);     // Domínio da página
console.log(document.charset);    // Codificação de caracteres (ex: "UTF-8")
console.log(document.documentElement); // O elemento <html>
console.log(document.head);       // O elemento <head>
console.log(document.body);       // O elemento <body>

// Modificando o título da página dinamicamente
document.title = "Novo Título da Aba!";
```

1.5 O Objeto `window`

O objeto `window` representa a janela do navegador e é o objeto global no contexto de um navegador. Isso significa que qualquer variável ou função declarada no escopo global se torna uma propriedade de `window`. O objeto `document` é, na verdade, uma propriedade de `window`.

```
// Estas duas linhas são equivalentes em um navegador
console.log(document === window.document); // true

// Propriedades do window
console.log(window.innerWidth); // Largura da janela do navegador (viewport)
console.log(window.innerHeight); // Altura da janela do navegador (viewport)
console.log(window.outerWidth); // Largura total da janela (incluindo barras)
console.log(window.scrollX); // Quantidade de pixels rolados horizontalmente
console.log(window.scrollY); // Quantidade de pixels rolados verticalmente
```

1.6 DOMContentLoaded vs. load

Um dos erros mais comuns de iniciantes é tentar manipular o DOM antes que ele esteja completamente construído. Se você colocar uma tag `<script>` no `<head>` do documento sem precauções, o JavaScript pode tentar acessar elementos que ainda não existem.

Existem duas abordagens principais para garantir que o DOM esteja pronto:

Abordagem 1: Colocar o script antes do `</body>`

Esta é a abordagem mais simples e recomendada para scripts que precisam interagir com o DOM. Como o HTML é analisado de cima para baixo, quando o script é executado, todos os elementos já foram processados.

```
<body>
  <h1>Conteúdo da Página</h1>
  <!-- Todo o conteúdo HTML aqui -->
  <script src="meu-script.js"></script>
</body>
```

Abordagem 2: Usar o evento `DOMContentLoaded`

O evento `DOMContentLoaded` é disparado no objeto `document` quando o documento HTML foi completamente carregado e analisado pelo navegador, sem esperar que folhas de estilo, imagens e subframes terminem de carregar.

```
document.addEventListener("DOMContentLoaded", function() {
  // Seguro para manipular o DOM aqui
  const h1 = document.querySelector("h1");
  h1.style.color = "blue";
  console.log("DOM pronto!");
});
```

O evento `load` (no objeto `window`) é disparado apenas quando a página inteira — incluindo todos os recursos dependentes como imagens, scripts e folhas de estilo — foi completamente carregada.

```
window.addEventListener("load", function() {
  // Todos os recursos foram carregados
  const imagem = document.querySelector("img");
  console.log(`Imagem carregada: ${imagem.naturalWidth}x${imagem.naturalHeight}px`);
});
```

A diferença de tempo entre `DOMContentLoaded` e `load` pode ser significativa em páginas com muitas imagens ou recursos externos. Para a maioria das manipulações de DOM, `DOMContentLoaded` é suficiente e mais rápido.

1.7 Atributos `defer` e `async` em Scripts

Uma alternativa moderna ao uso de `DOMContentLoaded` é usar os atributos `defer` ou `async` na tag `<script>` no `<head>`.

Atributo	Comportamento
Nenhum	O script bloqueia o parsing do HTML. Executa imediatamente.
<code>async</code>	O script é baixado em paralelo. Executa assim que o download termina (pode ser antes do DOM estar pronto).
<code>defer</code>	O script é baixado em paralelo. Executa apenas após o HTML ser completamente analisado, antes do <code>DOMContentLoaded</code> .

```
<head>
  <!-- Este script será executado após o DOM estar pronto -->
  <script src="meu-script.js" defer></script>
</head>
```

O uso de `defer` é geralmente a melhor prática para scripts que interagem com o DOM, pois combina o benefício de não bloquear o parsing com a garantia de que o DOM estará disponível quando o script executar.

Capítulo 2: Seleção de Elementos

2.1 Por que a Seleção é Fundamental?

Antes de poder alterar qualquer coisa na página, você precisa obter uma referência ao elemento que deseja modificar. O processo de encontrar e referenciar elementos no DOM é chamado de **seleção**. O JavaScript oferece múltiplos métodos para isso, cada um com suas características de uso, performance e compatibilidade.

2.2 `getElementById()`

O método `getElementById()` é o mais rápido e direto para encontrar um único elemento. Ele busca na árvore do DOM um elemento cujo atributo `id` seja exatamente igual ao valor fornecido. Como IDs devem ser únicos em um documento HTML, este método sempre retorna no máximo um elemento ou `null` se nenhum for encontrado.

```
// HTML: <div id="cabecalho">...</div>
const cabecalho = document.getElementById("cabecalho");

if (cabecalho !== null) {
  console.log("Elemento encontrado:", cabecalho);
  cabecalho.style.backgroundColor = "lightblue";
} else {
  console.log("Elemento não encontrado.");
}
```

Importante: `getElementById()` é chamado apenas no objeto `document`, não em outros elementos. Não existe `elemento.getElementById()`.

2.3 `getElementsByClassName()`

Este método retorna uma `HTMLCollection` (viva) de todos os elementos que possuem a classe CSS especificada. O termo "viva" significa que a coleção é atualizada automaticamente se elementos com aquela classe forem adicionados ou removidos do DOM.

```
// HTML: <p class="destaque">...</p> <span class="destaque">...</span>
const elementosDestaque = document.getElementsByClassName("destaque");

console.log(elementosDestaque.length); // Número de elementos encontrados

// Iterando com um loop for clássico (HTMLCollection não tem forEach nativo)
for (let i = 0; i < elementosDestaque.length; i++) {
    elementosDestaque[i].style.fontWeight = "bold";
}

// Convertendo para Array para usar métodos modernos
const arrayDestaque = Array.from(elementosDestaque);
arrayDestaque.forEach(el => el.classList.add("processado"));
```

Você também pode buscar por múltiplas classes separando-as por espaço:

```
// Retorna elementos que têm AMBAS as classes "card" e "ativo"
const cardsAtivos = document.getElementsByClassName("card ativo");
```

2.4 `getElementsByTagName()`

Retorna uma `HTMLCollection` viva de todos os elementos com o nome de tag especificado.

```
// Seleciona todos os parágrafos
const todosOsParagrafos = document.getElementsByTagName("p");

// Seleciona todos os elementos (usando "*")
const todosOsElementos = document.getElementsByTagName("*");
console.log(`Total de elementos na página: ${todosOsElementos.length}`);
```

2.5 `querySelector()` — O Seletor Moderno

O método `querySelector()` aceita qualquer seletor CSS válido como argumento e retorna o **primeiro** elemento que corresponde a esse seletor. Se nenhum elemento for encontrado, retorna `null`. Este método pode ser chamado tanto no `document` quanto em qualquer elemento, limitando a busca à subárvore daquele elemento.

```
// Seletor de ID
const titulo = document.querySelector("#titulo-principal");

// Seletor de classe
const primeiroCard = document.querySelector(".card");

// Seletor de tag
const primeiraImagem = document.querySelector("img");

// Seletor de atributo
const inputEmail = document.querySelector("input[type='email']");

// Seletor descendente
const linkNoMenu = document.querySelector("nav ul li a");

// Pseudo-classe
const primeiroLi = document.querySelector("li:first-child");

// Busca dentro de um elemento específico
const formulario = document.querySelector("#form-login");
const senhaInput = formulario.querySelector("input[type='password']");
```

2.6 `querySelectorAll()` — Selecionando Múltiplos Elementos

`querySelectorAll()` funciona como `querySelector()`, mas retorna um `NodeList` **estático** contendo todos os elementos que correspondem ao seletor. Por ser estático, ele não é atualizado automaticamente quando o DOM muda.

```
// Seleciona todos os itens de lista
const todosOsItens = document.querySelectorAll("li");

// NodeList tem suporte nativo a forEach
todosOsItens.forEach(function(item, indice) {
    console.log(`Item ${indice}: ${item.textContent}`);
});

// Seletores complexos
const botoesPrimarios = document.querySelectorAll(".btn.btn-primary");
const inputsObrigatorios = document.querySelectorAll("input[required]");
const paragrafosNaoOcultos = document.querySelectorAll("p:not(.oculto)");
```

2.7 HTMLCollection vs. NodeList

Entender a diferença entre `HTMLCollection` e `NodeList` é importante para evitar bugs sutis:

Característica	HTMLCollection	NodeList
Retornado por	<code>getElementsByClassName()</code> , <code>getElementsByName()</code> , <code>children</code>	<code>querySelectorAll()</code> , <code>childNodes</code>
Conteúdo	Apenas nós de Elemento	Pode conter qualquer tipo de nó
Atualização	Viva (atualiza com o DOM)	Estática (snapshot do momento da chamada)
<code>forEach()</code>	Não suportado nativamente	Suportado nativamente
Acesso por nome	Sim (por <code>id</code> ou <code>name</code>)	Não

```
const lista = document.querySelector("ul");

// HTMLCollection (viva)
const filhosElementos = lista.children;

// NodeList estático
const filhosQuery = lista.querySelectorAll("li");

// Adicionando um novo item
const novoItem = document.createElement("li");
novoItem.textContent = "Novo Item";
lista.appendChild(novoItem);

// filhosElementos agora inclui o novo item (é viva)
console.log(filhosElementos.length); // Aumentou em 1

// filhosQuery NÃO inclui o novo item (é estática)
console.log(filhosQuery.length); // Permanece o mesmo
```

2.8 Convertendo Coleções em Arrays

Para usar todos os métodos de array (`map`, `filter`, `reduce`, etc.) em uma `HTMLCollection` ou `NodeList`, converta-as para um array real:


```
const colecao = document.getElementsByTagName("p");

// Método 1: Array.from()
const arrayDeParagrafos = Array.from(colecao);

// Método 2: Spread operator
const arrayDeParagrafos2 = [...colecao];

// Agora você pode usar todos os métodos de array
const textosDeParagrafos = arrayDeParagrafos
  .filter(p => p.classList.contains("importante"))
  .map(p => p.textContent);

console.log(textosDeParagrafos);
```

2.9 Performance na Seleção de Elementos

A seleção de elementos tem um custo computacional. Em aplicações que manipulam o DOM intensamente, seguir boas práticas de performance faz diferença:

Armazene referências em variáveis: Se você vai usar um elemento várias vezes, não o selecione repetidamente. Selecione uma vez e guarde a referência.

```
// RUIM: Seleciona o elemento 3 vezes
document.querySelector("#meu-botao").style.color = "red";
document.querySelector("#meu-botao").textContent = "Clicado";
document.querySelector("#meu-botao").disabled = true;

// BOM: Seleciona uma vez e reutiliza
const botao = document.querySelector("#meu-botao");
botao.style.color = "red";
botao.textContent = "Clicado";
botao.disabled = true;
```

Limite o escopo da busca: Quando possível, chame `querySelector` em um elemento pai específico em vez de no `document` inteiro.

```
// Mais lento: busca em todo o documento
const item = document.querySelector(".lista-produtos .item");

// Mais rápido: busca apenas dentro do container
const listaProdutos = document.querySelector(".lista-produtos");
const item2 = listaProdutos.querySelector(".item");
```

Capítulo 3: Navegação na Árvore do DOM

3.1 Entendendo as Relações entre Nós

A árvore do DOM é uma estrutura hierárquica. Cada nó pode ter um pai (parent), filhos (children) e irmãos (siblings). Navegar por essas relações é essencial para muitas tarefas de manipulação do DOM.

3.2 Acessando o Pai: `parentNode` e `parentElement`

Ambas as propriedades retornam o nó pai de um elemento. A diferença é sutil:

- `parentNode`: Retorna o nó pai, que pode ser qualquer tipo de nó (incluindo `Document`).
- `parentElement`: Retorna o elemento pai, mas retorna `null` se o pai não for um nó do tipo `Element`.

```
const itemLista = document.querySelector("li");

// Ambos geralmente retornam o mesmo resultado
console.log(itemLista.parentNode); // <ul> ou <ol>
console.log(itemLista.parentElement); // <ul> ou <ol>

// Caso especial: o pai do <html> é o Document
const html = document.documentElement;
console.log(html.parentNode); // #document (o objeto Document)
console.log(html.parentElement); // null (Document não é um Element)
```

3.3 Acessando Filhos

O DOM oferece várias formas de acessar os filhos de um elemento:

Propriedade	Retorna	Inclui Nós de Texto?
<code>childNodes</code>	NodeList de todos os nós filhos	Sim
<code>children</code>	HTMLCollection de nós filhos do tipo Element	Não
<code>firstChild</code>	O primeiro nó filho	Sim
<code>lastChild</code>	O último nó filho	Sim
<code>firstElementChild</code>	O primeiro filho do tipo Element	Não
<code>lastElementChild</code>	O último filho do tipo Element	Não
<code>childElementCount</code>	Número de filhos do tipo Element	N/A

```
const lista = document.querySelector("ul");

// childNodes inclui nós de texto (espaços em branco entre as tags)
console.log(lista.childNodes);
// NodeList [text, li, text, li, text, li, text] (os "text" são os espaços)

// children inclui apenas elementos
console.log(lista.children);
// HTMLCollection [li, li, li]

// Acessando o primeiro e último elemento filho
console.log(lista.firstElementChild); // Primeiro <li>
console.log(lista.lastElementChild);  // Último <li>

// Contando filhos
console.log(lista.childElementCount); // 3
```

3.4 Navegando entre Irmãos

Para mover-se entre elementos no mesmo nível da hierarquia (irmãos), use:

Propriedade	Retorna
<code>nextSibling</code>	O próximo nó irmão (pode ser texto)
<code>previousSibling</code>	O nó irmão anterior (pode ser texto)
<code>nextElementSibling</code>	O próximo irmão do tipo Element
<code>previousElementSibling</code>	O irmão anterior do tipo Element

```
const segundoItem = document.querySelectorAll("li")[1];

// Irmão seguinte (elemento)
const terceiroItem = segundoItem.nextElementSibling;
console.log(terceiroItem.textContent);

// Irmão anterior (elemento)
const primeiroItem = segundoItem.previousElementSibling;
console.log(primeiroItem.textContent);
```

3.5 O Método `closest()`

O método `closest()` é extremamente útil. Ele percorre a árvore do DOM para cima (do elemento atual em direção à raiz) e retorna o **ancestral mais próximo** que corresponde ao seletor CSS fornecido. Se nenhum ancestral corresponder, retorna `null`.

```
// Estrutura HTML:
// <div class="card">
//   <div class="card-body">
//     <button class="btn-deletar">Deletar</button>
//   </div>
// </div>

document.addEventListener("click", function(e) {
  const btnDeletar = e.target.closest(".btn-deletar");

  if (btnDeletar) {
    // Encontra o card pai, não importa quão aninhado o botão esteja
    const card = btnDeletar.closest(".card");
    card.remove();
  }
});
```

3.6 O Método `contains()`

O método `contains()` verifica se um elemento é descendente de outro (ou o próprio elemento).

```
const container = document.querySelector(".container");
const botaoInterno = document.querySelector(".container .btn");

console.log(container.contains(botaoInterno)); // true
console.log(botaoInterno.contains(container)); // false
console.log(container.contains(container));    // true (um elemento contém a si mesmo)
```

3.7 Percorrendo o DOM Recursivamente

Às vezes, é necessário percorrer toda a subárvore de um elemento. Isso pode ser feito com recursão:

```
function percorrerDOM(no, nivel = 0) {  
  const indentacao = "  ".repeat(nivel);  
  
  if (no.nodeType === Node.ELEMENT_NODE) {  
    console.log(`${indentacao}<${no.tagName.toLowerCase()}>`);  
  } else if (no.nodeType === Node.TEXT_NODE && no.textContent.trim() !== "") {  
    console.log(`${indentacao}"${no.textContent.trim()}"`);  
  }  
  
  no.childNodes.forEach(filho => percorrerDOM(filho, nivel + 1));  
}  
  
percorrerDOM(document.body);
```

Capítulo 4: Manipulação de Conteúdo

4.1 As Três Propriedades Principais

Para alterar o conteúdo de um elemento, o JavaScript oferece três propriedades principais: `innerHTML`, `innerText` e `textContent`. Cada uma tem comportamentos distintos e casos de uso específicos.

4.2 `innerHTML`: Poder e Responsabilidade

A propriedade `innerHTML` permite ler ou definir o conteúdo HTML completo de um elemento, incluindo tags HTML.

```
const div = document.querySelector("#conteudo");

// Lendo o HTML interno
console.log(div.innerHTML);
// Saída: "<h2>Título</h2><p>Texto <strong>importante</strong></p>"

// Escrevendo HTML
div.innerHTML = "<h2>Novo Título</h2><p>Novo conteúdo com <em>ênfase</em>.</p>";

// Adicionando ao HTML existente (use com cuidado)
div.innerHTML += "<p>Parágrafo adicionado.</p>";
```

Aviso de Segurança (XSS): Nunca insira conteúdo fornecido pelo usuário diretamente via `innerHTML` sem sanitização. Isso pode abrir brechas de segurança do tipo Cross-Site Scripting (XSS).


```
// PERIGOSO: Se userInput contiver "<script>alert('XSS')</script>", o script será executado
const userInput = obterInputDoUsuario();
div.innerHTML = userInput; // NÃO FAÇA ISSO!

// SEGURO: Use.textContent para texto puro
div.textContent = userInput; // Tags HTML serão exibidas como texto, não executadas
```

4.3 `textContent`: Seguro e Performático

A propriedade `textContent` obtém ou define o conteúdo de texto de um nó e de todos os seus descendentes. Ela ignora completamente as tags HTML — ao ler, retorna apenas o texto; ao escrever, trata o valor como texto puro (qualquer tag HTML será exibida como texto literal).

```
const paragrafo = document.querySelector("p");

// HTML: <p>Texto com <strong>negrito</strong> e <em>itálico</em>.</p>
console.log(paragrafo.textContent);
// Saída: "Texto com negrito e itálico."

// Escrevendo: tags HTML serão exibidas como texto
paragrafo.textContent = "Novo texto com <b>tags</b> que aparecem literalmente.";
// Resultado visual: Novo texto com <b>tags</b> que aparecem literalmente.
```

`textContent` é mais rápido que `innerHTML` porque não precisa analisar HTML. Use-o sempre que precisar apenas trabalhar com texto.

4.4 `innerText`: Consciente do CSS

`innerText` é semelhante ao `textContent`, mas com uma diferença crucial: ele é **consciente do estilo CSS**. Ele não retornará o texto de elementos que estão visualmente ocultos (com `display: none` ou `visibility: hidden`). Além disso, `innerText` respeita a formatação visual (quebras de linha, etc.).

```
// HTML:
// <div id="teste">
//   <p>Parágrafo visível</p>
//   <p style="display:none">Parágrafo oculto</p>
// </div>

const div = document.getElementById("teste");

console.log(div.textContent);
// "Parágrafo visível\nParágrafo oculto" (inclui o texto oculto)

console.log(div.innerText);
// "Parágrafo visível" (ignora o texto oculto)
```

A tabela a seguir resume as diferenças:

Característica	<code>innerHTML</code>	<code>textContent</code>	<code>innerText</code>
Retorna tags HTML	Sim	Não	Não
Executa scripts	Sim (em alguns casos)	Não	Não
Consciente do CSS	Não	Não	Sim
Performance	Mais lento	Mais rápido	Médio
Segurança	Risco de XSS	Seguro	Seguro

4.5 `outerHTML`: Incluindo o Próprio Elemento

Enquanto `innerHTML` acessa o conteúdo interno de um elemento, `outerHTML` inclui o próprio elemento na string HTML.

```
const link = document.querySelector("a");

// Lendo
console.log(link.innerHTML); // "Clique aqui"
console.log(link.outerHTML); // '<a href="https://exemplo.com">Clique aqui</a>'

// Escrevendo: substituí o elemento inteiro
link.outerHTML = "<button>Botão no lugar do link</button>";
// Atenção: a variável 'link' agora aponta para um elemento removido do DOM!
```

4.6 Trabalhando com Nós de Texto Diretamente

Para manipulação de texto de baixo nível, você pode trabalhar diretamente com nós de texto usando `createTextNode()` e a propriedade `nodeValue` ou `data`.

```
// Criando um nó de texto
const noDeTexto = document.createTextNode("Texto criado via JavaScript.");

// Adicionando ao DOM
const paragrafo = document.createElement("p");
paragrafo.appendChild(noDeTexto);
document.body.appendChild(paragrafo);

// Modificando o texto de um nó de texto existente
const h1 = document.querySelector("h1");
const textoDoH1 = h1.firstChild; // Assume que o primeiro filho é um nó de texto
textoDoH1.nodeValue = "Título Modificado";
// ou
textoDoH1.data = "Título Modificado (usando .data)";
```

Capítulo 5: Atributos, Propriedades e Dataset

5.1 A Diferença Crucial entre Atributos e Propriedades

Um dos conceitos mais confusos para iniciantes no DOM é a distinção entre **atributos HTML** e **propriedades do DOM**. Eles estão relacionados, mas são coisas diferentes.

Atributos são definidos no código HTML e representam o estado inicial do elemento. Eles são sempre strings.

Propriedades são características dos objetos JavaScript que representam os elementos no DOM. Elas podem ter qualquer tipo de dado (string, número, booleano, objeto).

Quando o navegador analisa o HTML, ele cria objetos DOM a partir das tags HTML e, para a maioria dos atributos padrão, cria propriedades correspondentes. No entanto, a sincronização entre atributos e propriedades não é sempre bidirecional.

```
const input = document.querySelector("input");

// O atributo "value" define o valor INICIAL
console.log(input.getAttribute("value")); // "valor inicial"

// A propriedade "value" reflete o valor ATUAL (pode mudar com a digitação do usuário)
console.log(input.value); // "valor inicial" (ou o que o usuário digitou)

// Após o usuário digitar "novo texto":
// input.getAttribute("value") ainda retorna "valor inicial"
// input.value retorna "novo texto"
```

5.2 Métodos para Manipulação de Atributos

O DOM fornece quatro métodos fundamentais para trabalhar com atributos:

```
const imagem = document.querySelector("img");

// getAttribute(nome): Retorna o valor do atributo como string, ou null se não existir
const srcAtual = imagem.getAttribute("src");
const altAtual = imagem.getAttribute("alt");
console.log(srcAtual, altAtual);

// setAttribute(nome, valor): Define ou cria um atributo
imagem.setAttribute("src", "nova-foto.jpg");
imagem.setAttribute("alt", "Descrição da nova foto");
imagem.setAttribute("width", "300");

// hasAttribute(nome): Retorna true se o atributo existir
if (imagem.hasAttribute("loading")) {
    console.log("A imagem tem o atributo loading");
}

// removeAttribute(nome): Remove o atributo completamente
imagem.removeAttribute("title");
```

5.3 Atributos Booleanos

Atributos booleanos HTML (como `disabled`, `checked`, `required`, `readonly`) funcionam de forma especial. A presença do atributo indica `true`; a ausência indica `false`. O valor do atributo em si não importa.

```
const botao = document.querySelector("button");
const checkbox = document.querySelector("input[type='checkbox']");

// Usando setAttribute/removeAttribute para atributos booleanos
botao.setAttribute("disabled", ""); // Desabilita o botão
botao.removeAttribute("disabled"); // Habilita o botão

// A forma mais simples: usar a propriedade diretamente
botao.disabled = true; // Desabilita
botao.disabled = false; // Habilita

checkbox.checked = true; // Marca o checkbox
checkbox.required = true; // Torna o campo obrigatório
```

5.4 Atributos `data-*`: Dados Customizados

Os atributos `data-*` foram introduzidos no HTML5 para permitir que desenvolvedores armazenem informações extras em elementos HTML sem usar atributos não padronizados ou truques como classes com dados.

```
<!-- No HTML -->
<article
  id="post-1"
  data-post-id="1"
  data-autor="João Silva"
  data-categoria="tecnologia"
  data-publicado="2026-01-15">
  <h2>Título do Post</h2>
</article>
```

```
const artigo = document.getElementById("post-1");

// Acessando via getAttribute (retorna string)
console.log(artigo.getAttribute("data-post-id")); // "1"

// Acessando via dataset (mais elegante)
// Atributos com hifens são convertidos para camelCase no dataset
console.log(artigo.dataset.postId); // "1"
console.log(artigo.dataset.autor); // "João Silva"
console.log(artigo.dataset.categoria); // "tecnologia"

// Modificando um atributo data
artigo.dataset.categoria = "programacao";

// Adicionando um novo atributo data
artigo.dataset.visualizacoes = "1500";
// Isso cria o atributo data-visualizacoes="1500" no HTML

// Removendo um atributo data
delete artigo.dataset.publicado;
// Isso remove o atributo data-publicado do HTML

// Verificando se um atributo data existe
if ("postId" in artigo.dataset) {
    console.log("O post tem um ID.");
}
```

5.5 Atributos Especiais: `href`, `src`, `action`

Alguns atributos como `href` e `src` têm comportamento especial. A propriedade DOM retorna a URL absoluta, enquanto `getAttribute()` retorna o valor exato como escrito no HTML.

```
const link = document.querySelector("a");
// HTML: <a href="/sobre">Sobre</a>

console.log(link.href); // "https://meusite.com/sobre" (URL absoluta)
console.log(link.getAttribute("href")); // "/sobre" (valor original)
```

5.6 Tabela de Atributos e Propriedades Comuns

Atributo HTML	Propriedade DOM	Tipo da Propriedade
<code>id</code>	<code>element.id</code>	String
<code>class</code>	<code>element.className</code>	String
<code>href</code>	<code>element.href</code>	String (URL absoluta)
<code>src</code>	<code>element.src</code>	String (URL absoluta)
<code>value</code>	<code>element.value</code>	String
<code>checked</code>	<code>element.checked</code>	Boolean
<code>disabled</code>	<code>element.disabled</code>	Boolean
<code>required</code>	<code>element.required</code>	Boolean
<code>readonly</code>	<code>element.readOnly</code>	Boolean
<code>selected</code>	<code>element.selected</code>	Boolean
<code>type</code>	<code>element.type</code>	String
<code>name</code>	<code>element.name</code>	String
<code>placeholder</code>	<code>element.placeholder</code>	String
<code>action</code>	<code>element.action</code>	String (URL absoluta)
<code>method</code>	<code>element.method</code>	String

Capítulo 6: Classes e Estilos CSS

6.1 Manipulando Classes com `className`

A propriedade `className` reflete o atributo `class` do elemento como uma string. Ela é a forma mais antiga de manipular classes.

```
const div = document.querySelector("div");

// Lendo as classes
console.log(div.className); // "caixa destaque ativa"

// Definindo classes (sobrescreve todas as classes existentes!)
div.className = "nova-classe outra-classe";

// Adicionando uma classe sem perder as existentes (forma manual)
div.className += " mais-uma-classe";
```

O problema com `className` é que você precisa manipular uma string, o que é propenso a erros. A solução moderna é `classList`.

6.2 O Poderoso `classList`

A propriedade `classList` retorna um objeto `DOMTokenList` que fornece métodos convenientes para trabalhar com as classes de um elemento individualmente.

```
const elemento = document.querySelector(".meu-elemento");

// add(classe1, classe2, ...): Adiciona uma ou mais classes
elemento.classList.add("visivel");
elemento.classList.add("animado", "destacado"); // Adiciona múltiplas de uma vez

// remove(classe1, classe2, ...): Remove uma ou mais classes
elemento.classList.remove("oculto");
elemento.classList.remove("antigo", "obsoleto");

// toggle(classe, força?): Alterna a classe
// Se a classe existir, remove; se não existir, adiciona
elemento.classList.toggle("ativo");

// Com o segundo argumento (booleano):
// true: força adicionar a classe
// false: força remover a classe
const estaAtivo = true;
elemento.classList.toggle("ativo", estaAtivo);

// contains(classe): Verifica se a classe existe (retorna boolean)
if (elemento.classList.contains("destaque")) {
    console.log("O elemento está destacado!");
}

// replace(classeAntiga, classeNova): Substitui uma classe por outra
elemento.classList.replace("tema-claro", "tema-escuro");

// item(indice): Retorna a classe no índice especificado
console.log(elemento.classList.item(0)); // Primeira classe

// length: Número de classes
console.log(elemento.classList.length);
```

6.3 Manipulando Estilos Inline com `style`

A propriedade `style` de um elemento permite ler e escrever estilos CSS diretamente no atributo `style` do elemento (estilos inline). As propriedades CSS são acessadas em **camelCase** (sem hifens).

```
const caixa = document.querySelector(".caixa");

// Definindo estilos
caixa.style.backgroundColor = "#3498db";
caixa.style.color = "white";
caixa.style.padding = "20px";
caixa.style.borderRadius = "8px";
caixa.style.fontSize = "16px";
caixa.style.fontFamily = "Arial, sans-serif";
caixa.style.display = "flex";
caixa.style.justifyContent = "center";
caixa.style.alignItems = "center";

// Lendo um estilo inline
console.log(caixa.style.backgroundColor); // "rgb(52, 152, 219)"

// Removendo um estilo inline (definindo como string vazia)
caixa.style.backgroundColor = "";
```

6.4 Tabela de Conversão CSS para JavaScript

Propriedade CSS	Propriedade JavaScript
<code>background-color</code>	<code>backgroundColor</code>
<code>font-size</code>	<code>fontSize</code>
<code>font-family</code>	<code>fontFamily</code>
<code>margin-top</code>	<code>marginTop</code>
<code>padding-left</code>	<code>paddingLeft</code>
<code>border-radius</code>	<code>borderRadius</code>
<code>z-index</code>	<code>zIndex</code>
<code>flex-direction</code>	<code>flexDirection</code>
<code>justify-content</code>	<code>justifyContent</code>
<code>align-items</code>	<code>alignItems</code>
<code>text-decoration</code>	<code>textDecoration</code>
<code>box-shadow</code>	<code>boxShadow</code>
<code>pointer-events</code>	<code>pointerEvents</code>
<code>max-width</code>	<code>maxWidth</code>
<code>min-height</code>	<code>minHeight</code>
<code>line-height</code>	<code>lineHeight</code>
<code>letter-spacing</code>	<code>letterSpacing</code>
<code>overflow-x</code>	<code>overflowX</code>
<code>list-style-type</code>	<code>listStyleType</code>
<code>text-transform</code>	<code>textTransform</code>

6.5 Lendo Estilos Computados com `getComputedStyle()`

A propriedade `style` só lê e escreve estilos **inline**. Para obter o estilo final que o navegador realmente aplicou a um elemento (após considerar todas as regras CSS de folhas de estilo, herança, etc.), use `window.getComputedStyle()`.

```
const titulo = document.querySelector("h1");

// Obtém o objeto de estilos computados
const estilos = window.getComputedStyle(titulo);

// Lendo propriedades computadas
console.log(estilos.fontSize);      // Ex: "32px"
console.log(estilos.color);        // Ex: "rgb(33, 37, 41)"
console.log(estilos.fontWeight);   // Ex: "700"
console.log(estilos.display);       // Ex: "block"
console.log(estilos.marginBottom); // Ex: "16px"

// Também funciona com pseudo-elementos
const antes = window.getComputedStyle(titulo, "::before");
console.log(antes.content);
```

Importante: `getComputedStyle()` retorna um objeto somente leitura. Você não pode definir propriedades nele.

6.6 Variáveis CSS (Custom Properties) com JavaScript

As variáveis CSS (Custom Properties) podem ser lidas e modificadas via JavaScript, permitindo mudanças de tema dinâmicas de forma eficiente.

```
/* No CSS */

:root {
  --cor-primaria: #3498db;
  --tamanho-fonte: 16px;
  --espacamento: 8px;
}
```

```
const raiz = document.documentElement; // O elemento :root

// Lendo uma variável CSS
const corPrimaria = getComputedStyle(raiz).getPropertyValue("--cor-primaria");
console.log(corPrimaria.trim()); // "#3498db"

// Definindo/modificando uma variável CSS
raiz.style.setProperty("--cor-primaria", "#e74c3c");
raiz.style.setProperty("--tamanho-fonte", "18px");

// Removendo uma variável CSS
raiz.style.removeProperty("--espacamento");
```

Capítulo 7: Criação e Inserção de Elementos

7.1 Criando Elementos com `createElement()`

O método `document.createElement(tagName)` cria um novo elemento HTML em memória (sem adicioná-lo ao DOM ainda).

```
// Criando diferentes tipos de elementos
const novaDiv = document.createElement("div");
const novoParagrafo = document.createElement("p");
const novoLink = document.createElement("a");
const novaImagem = document.createElement("img");
const novoInput = document.createElement("input");

// Configurando o elemento antes de inserir no DOM
novoLink.href = "https://www.exemplo.com";
novoLink.textContent = "Visite nosso site";
novoLink.target = "_blank";
novoLink.classList.add("link-externo");

novaImagem.src = "foto.jpg";
novaImagem.alt = "Descrição da foto";
novaImagem.width = 300;

novoInput.type = "email";
novoInput.placeholder = "Digite seu e-mail";
novoInput.required = true;
novoInput.classList.add("form-control");
```

7.2 Inserindo Elementos com `appendChild()` e `append()`

Após criar e configurar um elemento, você precisa inseri-lo na árvore do DOM.

`appendChild(no)` insere um nó como o **último filho** do elemento pai. Ele aceita apenas um nó por vez.

```
const container = document.getElementById("container");
const novoParagrafo = document.createElement("p");
novoParagrafo.textContent = "Parágrafo adicionado ao final.";

container.appendChild(novoParagrafo);
```

`append(...nos)` é o método moderno. Ele pode inserir múltiplos nós e strings de texto de uma vez, como o último filho.

```
const lista = document.querySelector("ul");

const item1 = document.createElement("li");
item1.textContent = "Item 1";

const item2 = document.createElement("li");
item2.textContent = "Item 2";

// Insere múltiplos elementos de uma vez
lista.append(item1, item2);

// Também pode inserir strings diretamente
lista.append("Texto simples"); // Cria um nó de texto
```

`prepend(...nos)` insere como o **primeiro filho**:

```
const lista2 = document.querySelector("ul");
const itemInicio = document.createElement("li");
itemInicio.textContent = "Item no início";
lista2.prepend(itemInicio);
```

7.3 Inserindo em Posições Específicas com `insertBefore()`

Para inserir um elemento antes de um irmão específico, use `insertBefore(novoNo, noReferencia)` no elemento pai.

```
const lista = document.querySelector("ul");
const itens = lista.querySelectorAll("li");

// Insere antes do terceiro item (índice 2)
const novoItem = document.createElement("li");
novoItem.textContent = "Item inserido antes do terceiro";
lista.insertBefore(novoItem, itens[2]);
```

7.4 `insertAdjacentHTML()`, `insertAdjacentElement()` e `insertAdjacentText()`

Estes métodos oferecem controle preciso sobre onde inserir conteúdo em relação a um elemento de referência, usando quatro posições possíveis:

Posição	Descrição
<code>"beforebegin"</code>	Antes do próprio elemento (como irmão anterior)
<code>"afterbegin"</code>	Dentro do elemento, antes do primeiro filho
<code>"beforeend"</code>	Dentro do elemento, após o último filho
<code>"afterend"</code>	Após o próprio elemento (como próximo irmão)

```
const paragrafo = document.querySelector("#meu-paragrafo");

// Insere HTML antes do elemento
paragrafo.insertAdjacentHTML("beforebegin", "<h3>Subtítulo antes</h3>");

// Insere HTML como primeiro filho
paragrafo.insertAdjacentHTML("afterbegin", "<strong>Início: </strong>");

// Insere HTML como último filho
paragrafo.insertAdjacentHTML("beforeend", " <em>(fim)</em>");

// Insere HTML após o elemento
paragrafo.insertAdjacentHTML("afterend", "<p>Parágrafo após</p>");
```

7.5 Clonando Elementos com `cloneNode()`

O método `cloneNode(deep)` cria uma cópia do elemento. Se `deep` for `true`, todos os descendentes também são copiados.

```
const cardOriginal = document.querySelector(".card");

// Clone superficial: copia apenas o elemento, sem filhos
const cloneSuperficial = cardOriginal.cloneNode(false);

// Clone profundo: copia o elemento e todos os seus descendentes
const cloneProfundo = cardOriginal.cloneNode(true);

// Modificando o clone antes de inserir
cloneProfundo.querySelector(".card-titulo").textContent = "Título do Clone";
cloneProfundo.id = "card-clone"; // IDs devem ser únicos!

document.querySelector(".grid").appendChild(cloneProfundo);
```

7.6 `DocumentFragment`: Inserções em Lote Performáticas

Quando você precisa inserir muitos elementos de uma vez, inserir um por um no DOM é ineficiente porque cada inserção pode causar um "reflow" (recálculo do layout). A solução é usar um `DocumentFragment`, que é um nó leve que existe fora da árvore do DOM principal.

```
const lista = document.querySelector("#lista-grande");
const fragment = document.createDocumentFragment();

// Criamos 1000 itens e adicionamos ao fragment (sem tocar no DOM real)
for (let i = 1; i <= 1000; i++) {
  const item = document.createElement("li");
  item.textContent = `Item número ${i}`;
  fragment.appendChild(item);
}

// Uma única inserção no DOM real (apenas um reflow)
lista.appendChild(fragment);
```

Capítulo 8: Remoção e Substituição de Elementos

8.1 Removendo Elementos com `remove()`

O método moderno e mais simples para remover um elemento do DOM é chamar `remove()` diretamente no elemento que você deseja excluir.

```
// Remove um elemento específico
const aviso = document.querySelector(".aviso-temporario");
if (aviso) {
  aviso.remove();
}

// Removendo após um tempo
const notificacao = document.querySelector(".notificacao");
setTimeout(() => {
  notificacao.remove();
}, 3000); // Remove após 3 segundos
```

8.2 Removendo com `removeChild()`

O método clássico `removeChild(no)` é chamado no elemento **pai** e remove o nó filho especificado. É útil quando você tem a referência do pai mas não do filho diretamente.

```
const lista = document.querySelector("ul");

// Remove o primeiro item
const primeiroItem = lista.firstChild;
lista.removeChild(primeiroItem);

// Remove o último item
const ultimoItem = lista.lastElementChild;
lista.removeChild(ultimoItem);

// Remove um item pelo índice
const indice = 2;
lista.removeChild(lista.children[indice]);
```

8.3 Substituindo Elementos

Para substituir um elemento por outro, use `replaceWith()` (moderno) ou `replaceChild()` (clássico).

```
// replaceWith(): chamado no elemento a ser substituído
const velhoTitulo = document.querySelector("h1");
const novoTitulo = document.createElement("h2");
novoTitulo.textContent = "Novo Subtítulo";
novoTitulo.classList.add("titulo-principal");
velhoTitulo.replaceWith(novoTitulo);

// replaceWith() também aceita strings
const velhoLink = document.querySelector("a.externo");
velhoLink.replaceWith("Texto simples no lugar do link");

// replaceChild(): chamado no pai
const container = document.querySelector(".container");
const elementoAntigo = container.querySelector(".antigo");
const elementoNovo = document.createElement("div");
elementoNovo.className = "novo";
elementoNovo.textContent = "Conteúdo novo";
container.replaceChild(elementoNovo, elementoAntigo);
```

8.4 Esvaziando um Elemento

Existem várias formas de remover todos os filhos de um elemento:


```
const container = document.getElementById("resultado");

// Método 1: innerHTML (mais simples, mas recria o DOM interno)
container.innerHTML = "";

// Método 2: textContent (mais rápido que innerHTML para esvaziamento)
container.textContent = "";

// Método 3: Loop com removeChild (mais verboso, mas explícito)
while (container.firstChild) {
    container.removeChild(container.firstChild);
}

// Método 4: replaceChildren() (moderno, substitui todos os filhos)
container.replaceChildren(); // Sem argumentos, apenas esvazia
```

Capítulo 9: Introdução a Eventos

9.1 O Modelo de Eventos do DOM

Eventos são o mecanismo pelo qual o JavaScript responde a interações do usuário e a ocorrências no navegador. Um evento é um sinal de que algo aconteceu — o usuário clicou em um botão, o mouse se moveu, uma tecla foi pressionada, a página terminou de carregar, etc.

O modelo de eventos do DOM é baseado no padrão Observer (Observador): você registra funções (chamadas de "listeners" ou "handlers") que serão chamadas quando um evento específico ocorrer em um elemento específico.

9.2 As Três Formas de Registrar Eventos

Forma 1: Atributos HTML Inline (Antipadrão)

Esta abordagem mistura HTML e JavaScript, tornando o código difícil de manter. Deve ser evitada em projetos modernos.

```
<!-- NÃO RECOMENDADO -->
<button onclick="alert('Clicado!')">Clique</button>
<input onchange="validar(this)" type="text">
```

Forma 2: Propriedades de Evento do DOM

Você pode atribuir uma função diretamente à propriedade de evento do elemento. A limitação é que só pode haver um manipulador por tipo de evento por elemento.

```
const botao = document.querySelector("#btn");

botao.onclick = function() {
    console.log("Clicado!");
};

// Se você atribuir novamente, o anterior é sobrescrito
botao.onclick = function() {
    console.log("Apenas este será executado.");
};

// Para remover: atribua null
botao.onclick = null;
```

Forma 3: `addEventListener()` (Recomendado)

Esta é a forma moderna e mais poderosa. Permite múltiplos manipuladores para o mesmo evento e oferece mais controle.

```
const botao = document.querySelector("#btn");

function manipulador1() {
    console.log("Primeiro manipulador");
}

function manipulador2() {
    console.log("Segundo manipulador");
}

// Ambos serão executados quando o botão for clicado
botao.addEventListener("click", manipulador1);
botao.addEventListener("click", manipulador2);

// Removendo um manipulador específico
botao.removeEventListener("click", manipulador1);
// Agora apenas manipulador2 será executado
```

9.3 O Objeto Event

Quando um evento é disparado, o navegador cria automaticamente um objeto `Event` e o passa como argumento para o manipulador. Este objeto contém informações detalhadas sobre o evento.

```
document.querySelector("#btn").addEventListener("click", function(event) {  
    // Tipo do evento  
    console.log(event.type);        // "click"  
  
    // Elemento que disparou o evento  
    console.log(event.target);      // O elemento clicado  
  
    // Elemento ao qual o listener foi anexado  
    console.log(event.currentTarget); // Pode ser diferente de target na delegação  
  
    // Timestamp do evento (milissegundos desde a origem do tempo)  
    console.log(event.timeStamp);  
  
    // Se o evento pode fazer bubble  
    console.log(event.bubbles);     // true para a maioria dos eventos  
  
    // Se o evento pode ser cancelado  
    console.log(event.cancelable);  // true para a maioria dos eventos  
});
```

9.4 `preventDefault()`: Cancelando o Comportamento Padrão

Muitos eventos têm comportamentos padrão do navegador. `preventDefault()` cancela esse comportamento padrão, mas não impede a propagação do evento.

```
// Evitar que um link navegue para outra página
const link = document.querySelector("a");
link.addEventListener("click", function(e) {
    e.preventDefault();
    console.log("Link clicado, mas navegação cancelada.");
});

// Evitar o envio padrão de um formulário
const form = document.querySelector("form");
form.addEventListener("submit", function(e) {
    e.preventDefault();
    // Processar os dados via JavaScript (AJAX/Fetch)
    console.log("Formulário interceptado.");
});

// Evitar o menu de contexto do botão direito
document.addEventListener("contextmenu", function(e) {
    e.preventDefault();
    // Mostrar um menu customizado
});
```

9.5 Propagação de Eventos: Bubbling e Capturing

Quando um evento é disparado em um elemento, ele não fica restrito a esse elemento. Ele percorre a árvore do DOM em três fases:

1. **Fase de Captura (Capture):** O evento desce do `window` até o elemento alvo.
2. **Fase de Alvo (Target):** O evento chega ao elemento que o disparou.
3. **Fase de Bubbling (Propagação):** O evento sobe do elemento alvo de volta ao `window`.

Por padrão, os manipuladores são registrados na fase de bubbling. Para registrar na fase de captura, passe `true` (ou `{ capture: true }`) como terceiro argumento do `addEventListener`.

```
const pai = document.querySelector(".pai");
const filho = document.querySelector(".filho");

// Fase de bubbling (padrão)
pai.addEventListener("click", function() {
    console.log("PAI clicado (bubbling)");
});

filho.addEventListener("click", function() {
    console.log("FILHO clicado (bubbling)");
});

// Ao clicar no filho, a saída será:
// "FILHO clicado (bubbling)"
// "PAI clicado (bubbling)"
// (o evento sobe do filho para o pai)

// Fase de captura
pai.addEventListener("click", function() {
    console.log("PAI clicado (captura)");
}, true); // true = fase de captura

// Ao clicar no filho agora, a saída será:
// "PAI clicado (captura)"
// "FILHO clicado (bubbling)"
// "PAI clicado (bubbling)"
```

9.6 `stopPropagation()` e `stopImmediatePropagation()`

`stopPropagation()` impede que o evento continue se propagando para os elementos pai (ou filho, na fase de captura).

`stopImmediatePropagation()` faz o mesmo, mas também impede que outros manipuladores do mesmo evento no mesmo elemento sejam executados.

```
const filho = document.querySelector(".filho");

filho.addEventListener("click", function(e) {
  e.stopPropagation(); // O evento não chegará ao pai
  console.log("Filho clicado, propagação parada.");
});

filho.addEventListener("click", function(e) {
  e.stopImmediatePropagation();
  console.log("Este é executado...");
});

filho.addEventListener("click", function(e) {
  console.log("...mas este NÃO será executado.");
});
```

9.7 Delegação de Eventos (Event Delegation)

A delegação de eventos é uma técnica poderosa que aproveita o bubbling. Em vez de adicionar um listener a cada elemento filho, você adiciona um único listener ao elemento pai. Quando um filho é clicado, o evento sobe até o pai, onde é capturado.

Vantagens:

- Menos listeners = melhor performance de memória.
- Funciona automaticamente para elementos adicionados dinamicamente ao DOM.

```

const lista = document.querySelector("#lista-tarefas");

// UM único listener no pai, em vez de N listeners nos filhos
lista.addEventListener("click", function(e) {
    const alvo = e.target;

    // Verifica qual elemento filho foi clicado
    if (alvo.classList.contains("btn-concluir")) {
        const tarefa = alvo.closest("li");
        tarefa.classList.toggle("concluida");
    }

    if (alvo.classList.contains("btn-excluir")) {
        const tarefa = alvo.closest("li");
        tarefa.remove();
    }
});

// Funciona mesmo para itens adicionados DEPOIS do listener ser registrado
function adicionarTarefa(texto) {
    const li = document.createElement("li");
    li.innerHTML = `
        <span>${texto}</span>
        <button class="btn-concluir">✓</button>
        <button class="btn-excluir">X</button>
    `;
    lista.appendChild(li);
}

```

9.8 Opções do `addEventListener()`

O terceiro argumento do `addEventListener()` pode ser um objeto de opções com mais controle:


```
const botao = document.querySelector("#btn");

botao.addEventListener("click", function() {
  console.log("Executado apenas uma vez!");
}, { once: true }); // Remove o listener automaticamente após a primeira execução

botao.addEventListener("scroll", function() {
  console.log("Scroll otimizado");
}, { passive: true }); // Indica que nunca chamará preventDefault() (melhora performance de scroll)
```

Capítulo 10: Eventos de Mouse e Pointer

10.1 A Família de Eventos de Mouse

O DOM oferece um conjunto rico de eventos para interações com o mouse. Cada evento tem seu momento específico de disparo e suas próprias propriedades.

10.2 Eventos de Clique

```
const botao = document.querySelector("#meu-botao");

// click: Botão esquerdo pressionado e solto no mesmo elemento
botao.addEventListener("click", function(e) {
    console.log("Clique simples!");
});

// dblclick: Dois cliques rápidos
botao.addEventListener("dblclick", function(e) {
    console.log("Clique duplo!");
});

// mousedown: Botão do mouse pressionado (antes de soltar)
botao.addEventListener("mousedown", function(e) {
    console.log(`Botão pressionado: ${e.button}`);
    // e.button: 0=esquerdo, 1=meio, 2=direito
});

// mouseup: Botão do mouse solto
botao.addEventListener("mouseup", function(e) {
    console.log("Botão solto!");
});
```

10.3 Eventos de Movimento e Hover

```
const caixa = document.querySelector(".caixa");

// mouseenter: Quando o mouse entra no elemento (NÃO faz bubble)
caixa.addEventListener("mouseenter", function() {
  this.style.backgroundColor = "lightblue";
  console.log("Mouse entrou na caixa");
});

// mouseleave: Quando o mouse sai do elemento (NÃO faz bubble)
caixa.addEventListener("mouseleave", function() {
  this.style.backgroundColor = "";
  console.log("Mouse saiu da caixa");
});

// mouseover: Quando o mouse entra no elemento OU em um filho (FAZ bubble)
caixa.addEventListener("mouseover", function(e) {
  console.log("mouseover em:", e.target.tagName);
});

// mouseout: Quando o mouse sai do elemento OU de um filho (FAZ bubble)
caixa.addEventListener("mouseout", function(e) {
  console.log("mouseout de:", e.target.tagName);
});

// mousemove: Disparado continuamente enquanto o mouse se move
caixa.addEventListener("mousemove", function(e) {
  const x = e.offsetX; // Coordenada X relativa ao elemento
  const y = e.offsetY; // Coordenada Y relativa ao elemento
  caixa.textContent = `X: ${x}, Y: ${y}`;
});
```

10.4 Coordenadas do Mouse

O objeto de evento de mouse fornece várias propriedades de coordenadas:

Propriedade	Descrição
<code>clientX</code> / <code>clientY</code>	Coordenadas relativas à viewport (área visível)
<code>pageX</code> / <code>pageY</code>	Coordenadas relativas ao documento inteiro (inclui scroll)
<code>screenX</code> / <code>screenY</code>	Coordenadas relativas à tela física do monitor
<code>offsetX</code> / <code>offsetY</code>	Coordenadas relativas ao elemento alvo do evento
<code>movementX</code> / <code>movementY</code>	Diferença de posição em relação ao último evento <code>mousemove</code>

```
document.addEventListener("mousemove", function(e) {  
  console.log(`  
    Client: (${e.clientX}, ${e.clientY})  
    Page:   (${e.pageX}, ${e.pageY})  
    Screen: (${e.screenX}, ${e.screenY})  
  `);  
});
```

10.5 Exemplo Prático: Tooltip Customizado

```
const tooltip = document.createElement("div");
tooltip.className = "tooltip";
tooltip.style.cssText = `
  position: fixed;
  background: #333;
  color: white;
  padding: 5px 10px;
  border-radius: 4px;
  font-size: 12px;
  pointer-events: none;
  display: none;
`;
document.body.appendChild(tooltip);

document.querySelectorAll("[data-tooltip]").forEach(el => {
  el.addEventListener("mouseenter", function(e) {
    tooltip.textContent = this.dataset.tooltip;
    tooltip.style.display = "block";
  });

  el.addEventListener("mousemove", function(e) {
    tooltip.style.left = (e.clientX + 15) + "px";
    tooltip.style.top = (e.clientY + 10) + "px";
  });

  el.addEventListener("mouseleave", function() {
    tooltip.style.display = "none";
  });
});
```

10.6 Pointer Events: A API Unificada

Os Pointer Events unificam mouse, toque e caneta em uma única API. São especialmente importantes para criar interfaces responsivas a diferentes tipos de entrada.

Evento de Pointer	Equivalente de Mouse
<code>pointerdown</code>	<code>mousedown</code> / <code>touchstart</code>
<code>pointerup</code>	<code>mouseup</code> / <code>touchend</code>
<code>pointermove</code>	<code>mousemove</code> / <code>touchmove</code>
<code>pointerenter</code>	<code>mouseenter</code>
<code>pointerleave</code>	<code>mouseleave</code>
<code>pointercancel</code>	—

```
const area = document.querySelector(".area-desenho");

area.addEventListener("pointerdown", function(e) {
  console.log(`Tipo de ponteiro: ${e.pointerType}`); // "mouse", "touch", "pen"
  console.log(`Pressão: ${e.pressure}`); // 0 a 1 (útil para canetas)
  console.log(`ID do ponteiro: ${e.pointerId}`); // Útil para multitouch
});
```

Capítulo 11: Eventos de Teclado

11.1 Os Eventos de Teclado

Existem dois eventos principais de teclado (o `keypress` foi depreciado e não deve ser usado):

- `keydown`: Disparado quando uma tecla é pressionada. Continua disparando repetidamente se a tecla for mantida pressionada.
- `keyup`: Disparado quando uma tecla é solta.

```
const input = document.querySelector("#campo-texto");

input.addEventListener("keydown", function(e) {
  console.log(`keydown: key="${e.key}", code="${e.code}"`);
});

input.addEventListener("keyup", function(e) {
  console.log(`keyup: key="${e.key}", code="${e.code}"`);
});
```

11.2 As Propriedades `key` e `code`

A propriedade `key` retorna o valor da tecla pressionada (o caractere que seria inserido, ou o nome da tecla especial). A propriedade `code` retorna o código físico da tecla no teclado, independente do layout ou idioma.

Tecla Pressionada	<code>e.key</code>	<code>e.code</code>
Letra "a" (minúscula)	<code>"a"</code>	<code>"KeyA"</code>
Letra "A" (maiúscula)	<code>"A"</code>	<code>"KeyA"</code>
Enter	<code>"Enter"</code>	<code>"Enter"</code>
Espaço	<code>" "</code>	<code>"Space"</code>
Seta para cima	<code>"ArrowUp"</code>	<code>"ArrowUp"</code>
Tecla 1 (teclado principal)	<code>"1"</code>	<code>"Digit1"</code>
Tecla 1 (teclado numérico)	<code>"1"</code>	<code>"Numpad1"</code>
Escape	<code>"Escape"</code>	<code>"Escape"</code>
Backspace	<code>"Backspace"</code>	<code>"Backspace"</code>
Tab	<code>"Tab"</code>	<code>"Tab"</code>
Shift	<code>"Shift"</code>	<code>"ShiftLeft"</code> ou <code>"ShiftRight"</code>

11.3 Teclas Modificadoras

O objeto de evento de teclado possui propriedades booleanas para verificar se teclas modificadoras estavam pressionadas:


```
document.addEventListener("keydown", function(e) {  
    if (e.ctrlKey) console.log("Ctrl está pressionado");  
    if (e.shiftKey) console.log("Shift está pressionado");  
    if (e.altKey) console.log("Alt está pressionado");  
    if (e.metaKey) console.log("Meta (Cmd/Win) está pressionado");  
  
    // Criando atalhos de teclado  
    if (e.ctrlKey && e.key === "s") {  
        e.preventDefault(); // Cancela o Ctrl+S do navegador  
        salvarDocumento();  
    }  
  
    if (e.ctrlKey && e.shiftKey && e.key === "Z") {  
        e.preventDefault();  
        refazer();  
    }  
});
```

11.4 Exemplo Prático: Navegação por Teclado

```
const itens = document.querySelectorAll(".item-lista");
let indiceFocado = -1;

document.addEventListener("keydown", function(e) {
  if (e.key === "ArrowDown") {
    e.preventDefault();
    indiceFocado = Math.min(indiceFocado + 1, itens.length - 1);
    atualizarFoco();
  }

  if (e.key === "ArrowUp") {
    e.preventDefault();
    indiceFocado = Math.max(indiceFocado - 1, 0);
    atualizarFoco();
  }

  if (e.key === "Enter" && indiceFocado >= 0) {
    itens[indiceFocado].click();
  }

  if (e.key === "Escape") {
    indiceFocado = -1;
    itens.forEach(item => item.classList.remove("focado"));
  }
});

function atualizarFoco() {
  itens.forEach((item, i) => {
    item.classList.toggle("focado", i === indiceFocado);
  });
  if (indiceFocado >= 0) {
    itens[indiceFocado].scrollIntoView({ block: "nearest" });
  }
}
```

Capítulo 12: Eventos de Formulário

12.1 O Evento `submit`

O evento `submit` é disparado no elemento `<form>` quando o formulário é enviado (seja clicando em um botão de submit ou pressionando Enter em um campo de texto). É o evento mais importante para trabalhar com formulários.

```
const form = document.querySelector("#form-contato");

form.addEventListener("submit", function(e) {
  e.preventDefault(); // Impede o recarregamento da página

  // Coletando dados do formulário
  const nome = form.querySelector("#nome").value;
  const email = form.querySelector("#email").value;
  const mensagem = form.querySelector("#mensagem").value;

  // Validação básica
  if (!nome || !email || !mensagem) {
    exibirErro("Por favor, preencha todos os campos.");
    return;
  }

  // Enviando os dados (exemplo com Fetch API)
  enviarFormulario({ nome, email, mensagem });
});
```

12.2 A API `FormData`

A classe `FormData` fornece uma maneira fácil de construir um conjunto de pares chave/valor representando os campos do formulário.

```
form.addEventListener("submit", function(e) {  
  e.preventDefault();  
  
  const dados = new FormData(form);  
  
  // Acessando valores individuais  
  console.log(dados.get("nome"));  
  console.log(dados.get("email"));  
  
  // Iterando sobre todos os campos  
  for (const [chave, valor] of dados.entries()) {  
    console.log(`${chave}: ${valor}`);  
  }  
  
  // Enviando com Fetch  
  fetch("/api/contato", {  
    method: "POST",  
    body: dados // FormData pode ser enviado diretamente  
  });  
});
```

12.3 Eventos `input` e `change`

```
const campoNome = document.querySelector("#nome");

// input: Disparado a CADA alteração (cada tecla digitada, colagem, etc.)
campoNome.addEventListener("input", function(e) {
  const valor = e.target.value;
  console.log(`Digitando: "${valor}"`);

  // Atualização em tempo real (ex: contador de caracteres)
  document.querySelector("#contador").textContent = `${valor.length}/100`;
});

// change: Disparado quando o valor muda E o elemento perde o foco
campoNome.addEventListener("change", function(e) {
  console.log(`Valor final confirmado: "${e.target.value}"`);
});
```

12.4 Eventos de Foco

```
const input = document.querySelector("input");

// focus: O elemento ganhou foco (NÃO faz bubble)
input.addEventListener("focus", function() {
  this.parentElement.classList.add("campo-ativo");
});

// blur: O elemento perdeu foco (NÃO faz bubble)
input.addEventListener("blur", function() {
  this.parentElement.classList.remove("campo-ativo");
  validarCampo(this);
});

// focusin: Igual ao focus, mas FAZ bubble (útil para delegação)
document.querySelector("form").addEventListener("focusin", function(e) {
  console.log(`Campo focado: ${e.target.name}`);
});

// focusout: Igual ao blur, mas FAZ bubble
document.querySelector("form").addEventListener("focusout", function(e) {
  console.log(`Campo desfocado: ${e.target.name}`);
});
```

12.5 Manipulando Checkboxes, Radios e Selects

```
// Checkbox
const checkbox = document.querySelector("#aceito-termos");
checkbox.addEventListener("change", function() {
    const botaoEnviar = document.querySelector("#btn-enviar");
    botaoEnviar.disabled = !this.checked;
    console.log(`Termos aceitos: ${this.checked}`);
});

// Radio buttons
const radios = document.querySelectorAll("input[name='genero']");
radios.forEach(radio => {
    radio.addEventListener("change", function() {
        console.log(`Gênero selecionado: ${this.value}`);
    });
});

// Select
const selectEstado = document.querySelector("#estado");
selectEstado.addEventListener("change", function() {
    const valorSelecionado = this.value;
    const textoSelecionado = this.options[this.selectedIndex].text;
    console.log(`Estado: ${textoSelecionado} (${valorSelecionado})`);

    // Carregar cidades do estado selecionado
    carregarCidades(valorSelecionado);
});
```

12.6 Validação de Formulários com a Constraint Validation API

O HTML5 introduziu a Constraint Validation API, que permite validar campos de formulário tanto via atributos HTML quanto via JavaScript.

```
const form = document.querySelector("#form-registro");
const emailInput = form.querySelector("#email");

// Verificando a validade de um campo
emailInput.addEventListener("blur", function() {
  if (!this.validity.valid) {
    if (this.validity.valueMissing) {
      this.setCustomValidity("O e-mail é obrigatório.");
    } else if (this.validity.typeMismatch) {
      this.setCustomValidity("Por favor, insira um e-mail válido.");
    }
    this.reportValidity(); // Exibe a mensagem de erro nativa
  } else {
    this.setCustomValidity(""); // Limpa o erro customizado
  }
});

// Propriedades do objeto ValidityState
// validity.valid: true se o campo é válido
// validity.valueMissing: true se o campo required está vazio
// validity.typeMismatch: true se o valor não corresponde ao tipo (ex: email inválido)
// validity.patternMismatch: true se o valor não corresponde ao pattern
// validity.toolong: true se o valor excede maxlength
// validity.tooshort: true se o valor é menor que minlength
// validity.rangeoverflow: true se o valor é maior que max
// validity.rangeunderflow: true se o valor é menor que min
```


Capítulo 13: Eventos de Janela e Documento

13.1 Ciclo de Vida da Página

O carregamento de uma página web passa por várias etapas, cada uma com seu evento correspondente:

Evento	Objeto	Quando Dispara
<code>DOMContentLoaded</code>	<code>document</code>	HTML analisado, DOM construído (sem aguardar recursos externos)
<code>load</code>	<code>window</code>	Todos os recursos (imagens, CSS, scripts) carregados
<code>beforeunload</code>	<code>window</code>	Antes da página ser descarregada (usuário saindo)
<code>unload</code>	<code>window</code>	Página sendo descarregada
<code>pageshow</code>	<code>window</code>	Página exibida (inclui navegação pelo histórico)
<code>pagehide</code>	<code>window</code>	Página sendo ocultada (inclui navegação pelo histórico)

```
// DOMContentLoaded: mais rápido, suficiente para manipulação do DOM
document.addEventListener("DOMContentLoaded", function() {
  console.log("DOM pronto!");
  inicializarApp();
});

// load: aguarda todos os recursos
window.addEventListener("load", function() {
  console.log("Página completamente carregada!");
  const imagem = document.querySelector("img");
  console.log(`Dimensões da imagem: ${imagem.naturalWidth}x${imagem.naturalHeight}`);
});

// beforeunload: aviso antes de sair
window.addEventListener("beforeunload", function(e) {
  if (temDadosNaoSalvos()) {
    e.preventDefault();
    e.returnValue = ""; // Necessário para alguns navegadores
    // O navegador exibirá uma caixa de diálogo padrão
  }
});
```

13.2 Evento `resize` e Debouncing

O evento `resize` é disparado na janela quando ela é redimensionada. Como ele pode disparar dezenas de vezes por segundo durante o redimensionamento, é essencial usar técnicas de otimização.

Debouncing garante que a função seja executada apenas após um período de inatividade:

```
function debounce(funcao, espera) {
  let temporizador;
  return function(...args) {
    clearTimeout(temporizador);
    temporizador = setTimeout(() => funcao.apply(this, args), espera);
  };
}

function aoRedimensionar() {
  console.log(`Nova largura: ${window.innerWidth}px`);
  ajustarLayout();
}

// A função só será chamada 300ms após o usuário parar de redimensionar
window.addEventListener("resize", debounce(aoRedimensionar, 300));
```

Throttling garante que a função seja executada no máximo uma vez por intervalo de tempo:

```
function throttle(funcao, limite) {
  let emEspera = false;
  return function(...args) {
    if (!emEspera) {
      funcao.apply(this, args);
      emEspera = true;
      setTimeout(() => { emEspera = false; }, limite);
    }
  };
}

// A função será chamada no máximo uma vez a cada 100ms
window.addEventListener("resize", throttle(aoRedimensionar, 100));
```

13.3 Evento `scroll`

O evento `scroll` é disparado quando o usuário rola a página ou um elemento com overflow.

```
// Detectando a posição do scroll
window.addEventListener("scroll", function() {
    const scrollY = window.scrollY; // ou window.pageYOffset
    const scrollX = window.scrollX;
    const alturaDocumento = document.documentElement.scrollHeight;
    const alturaViewport = window.innerHeight;
    const percentualRolado = (scrollY / (alturaDocumento - alturaViewport)) * 100;

    // Barra de progresso de leitura
    document.querySelector(".barra-progresso").style.width = `${percentualRolado}%`;

    // Botão "Voltar ao topo"
    const btnTopo = document.querySelector(".btn-topo");
    btnTopo.style.display = scrollY > 300 ? "block" : "none";

    // Header fixo com sombra ao rolar
    const header = document.querySelector("header");
    header.classList.toggle("com-sombra", scrollY > 0);
});

// Rolando programaticamente
window.scrollTo({ top: 0, behavior: "smooth" }); // Volta ao topo suavemente
window.scrollBy({ top: 100, behavior: "smooth" }); // Rola 100px para baixo

// Rolando um elemento específico para a view
document.querySelector("#secao-contato").scrollIntoView({ behavior: "smooth" });
```

13.4 Eventos de Histórico e Roteamento

```
// hashchange: Disparado quando o fragmento da URL (#) muda
window.addEventListener("hashchange", function() {
    const hash = window.location.hash; // Ex: "#sobre"
    console.log(`Hash mudou para: ${hash}`);
    navegarParaSecao(hash.substring(1)); // Remove o "#"
});

// popstate: Disparado quando o histórico de navegação muda
window.addEventListener("popstate", function(e) {
    console.log("Estado:", e.state);
    renderizarPagina(window.location.pathname);
});

// Adicionando entradas ao histórico sem recarregar a página
history.pushState({ pagina: "sobre" }, "Sobre", "/sobre");
history.replaceState({ pagina: "home" }, "Home", "/");
```

13.5 Page Visibility API

```
// visibilitychange: Detecta quando o usuário muda de aba ou minimiza o navegador
document.addEventListener("visibilitychange", function() {
  if (document.hidden) {
    console.log("Usuário saiu da aba - pausando animações/timers");
    pausarVideo();
    pausarAnimacoes();
  } else {
    console.log("Usuário voltou para a aba - retomando");
    retomarVideo();
    retomarAnimacoes();
  }
});

// document.visibilityState: "visible" | "hidden" | "prerender"
console.log(document.visibilityState);
```

Capítulo 14: Eventos Avançados e Customizados

14.1 Criando Eventos Customizados

O JavaScript permite criar seus próprios tipos de eventos usando os construtores `Event` e `CustomEvent`. Isso é fundamental para criar componentes desacoplados que se comunicam via eventos.

```
// Evento simples (sem dados extras)
const eventoSimples = new Event("meuEvento", {
  bubbles: true,    // O evento fará bubble
  cancelable: true  // O evento pode ser cancelado com preventDefault()
});

// Evento customizado (com dados extras no detalhe)
const eventoComDados = new CustomEvent("usuarioLogado", {
  bubbles: true,
  cancelable: false,
  detail: {
    id: 42,
    nome: "Maria Silva",
    email: "maria@exemplo.com",
    permissoes: ["leitura", "escrita"]
  }
});
```

14.2 Disparando e Ouvindo Eventos Customizados

```
// Sistema de autenticação desacoplado
const sistemaAuth = {
  login(usuario) {
    // Lógica de login...

    // Notifica o resto da aplicação via evento
    document.dispatchEvent(new CustomEvent("auth:login", {
      detail: { usuario }
    }));
  },

  logout() {
    document.dispatchEvent(new CustomEvent("auth:logout"));
  }
};

// Componente de Header ouve o evento
document.addEventListener("auth:login", function(e) {
  const { usuario } = e.detail;
  document.querySelector(".nome-usuario").textContent = usuario.nome;
  document.querySelector(".btn-login").style.display = "none";
  document.querySelector(".btn-logout").style.display = "block";
});

// Componente de Carrinho ouve o evento
document.addEventListener("auth:logout", function() {
  limparCarrinho();
  redirecionarParaLogin();
});
```

14.3 Eventos de Mídia

Elementos de mídia (`<video>` e `<audio>`) possuem um conjunto rico de eventos:

Evento	Quando Dispara
<code>play</code>	A reprodução começou
<code>pause</code>	A reprodução foi pausada
<code>ended</code>	A reprodução chegou ao fim
<code>timeupdate</code>	A posição de reprodução mudou (disparado frequentemente)
<code>volumechange</code>	O volume foi alterado
<code>loadedmetadata</code>	Os metadados (duração, dimensões) foram carregados
<code>canplay</code>	Há dados suficientes para iniciar a reprodução
<code>seeking</code>	O usuário está buscando uma posição
<code>seeked</code>	O usuário terminou de buscar
<code>error</code>	Ocorreu um erro

```
const video = document.querySelector("video");

// Criando um player customizado
video.addEventListener("timeupdate", function() {
    const progresso = (video.currentTime / video.duration) * 100;
    document.querySelector(".barra-progresso").style.width = `${progresso}%`;
    document.querySelector(".tempo-atual").textContent = formatarTempo(video.currentTime);
});

video.addEventListener("ended", function() {
    document.querySelector(".btn-play").textContent = "Assistir Novamente";
    sugerirProximoVideo();
});

function formatarTempo(segundos) {
    const min = Math.floor(segundos / 60);
    const seg = Math.floor(segundos % 60).toString().padStart(2, "0");
    return `${min}:${seg}`;
}
```

14.4 Eventos de Clipboard

```
// copy: Disparado quando o usuário copia conteúdo
document.addEventListener("copy", function(e) {
    const selecao = window.getSelection().toString();
    console.log(`Copiado: "${selecao}"`);

    // Modificar o que vai para a área de transferência
    e.clipboardData.setData("text/plain", selecao.toUpperCase());
    e.preventDefault();
});

// cut: Disparado quando o usuário recorta conteúdo
document.addEventListener("cut", function(e) {
    console.log("Conteúdo recortado");
});

// paste: Disparado quando o usuário cola conteúdo
document.addEventListener("paste", function(e) {
    const textoColar = e.clipboardData.getData("text/plain");
    console.log(`Colando: "${textoColar}"`);

    // Sanitizar antes de colar (ex: remover HTML)
    e.preventDefault();
    document.execCommand("insertText", false, textoColar);
});
```

Capítulo 15: O BOM e APIs Web Úteis

15.1 O Objeto `window` em Profundidade

O objeto `window` é o objeto global do JavaScript no contexto do navegador. Todas as variáveis globais e funções são propriedades de `window`.

```
// Propriedades de dimensão
console.log(window.innerWidth); // Largura da viewport
console.log(window.innerHeight); // Altura da viewport
console.log(window.outerWidth); // Largura total da janela
console.log(window.outerHeight); // Altura total da janela
console.log(window.devicePixelRatio); // Proporção de pixels do dispositivo

// Propriedades de posição do scroll
console.log(window.scrollX); // Pixels rolados horizontalmente
console.log(window.scrollY); // Pixels rolados verticalmente

// Métodos de diálogo
window.alert("Mensagem de alerta");
const confirmado = window.confirm("Você tem certeza?"); // Retorna boolean
const entrada = window.prompt("Digite seu nome:", "Padrão"); // Retorna string ou null

// Métodos de scroll
window.scrollTo(0, 0); // Vai para o topo
window.scrollTo({ top: 500, behavior: "smooth" });
window.scrollBy(0, 100); // Rola 100px para baixo
```

15.2 `setTimeout()` e `setInterval()`

```
// setTimeout: Executa uma função após um atraso
const idTimeout = setTimeout(function() {
    console.log("Executado após 2 segundos");
}, 2000);

// Cancelar o timeout antes de executar
clearTimeout(idTimeout);

// setInterval: Executa uma função repetidamente em um intervalo
let contador = 0;
const idInterval = setInterval(function() {
    contador++;
    console.log(`Contador: ${contador}`);

    if (contador >= 5) {
        clearInterval(idInterval); // Para após 5 execuções
    }
}, 1000);

// Exemplo prático: Relógio em tempo real
function atualizarRelogio() {
    const agora = new Date();
    const horas = agora.getHours().toString().padStart(2, "0");
    const minutos = agora.getMinutes().toString().padStart(2, "0");
    const segundos = agora.getSeconds().toString().padStart(2, "0");
    document.querySelector("#relogio").textContent = `${horas}:${minutos}:${segundos}`
}

atualizarRelogio(); // Executa imediatamente
setInterval(atualizarRelogio, 1000); // Atualiza a cada segundo
```

15.3 `requestAnimationFrame()` para Animações

`requestAnimationFrame()` é a forma correta de criar animações em JavaScript. Ele sincroniza a execução com a taxa de atualização do monitor (geralmente 60fps), resultando em animações mais suaves e eficientes.

```
const caixa = document.querySelector(".caixa-animada");
let posicao = 0;
let animacaoId;

function animar(timestamp) {
  posicao += 2; // Move 2px por frame
  caixa.style.transform = `translateX(${posicao}px)`;

  if (posicao < 500) {
    animacaoId = requestAnimationFrame(animar); // Solicita o próximo frame
  }
}

// Inicia a animação
animacaoId = requestAnimationFrame(animar);

// Para a animação
// cancelAnimationFrame(animacaoId);
```

15.4 O Objeto `navigator`

```
// Informações do navegador e sistema
console.log(navigator.userAgent); // String de identificação do navegador
console.log(navigator.language); // Idioma preferido (ex: "pt-BR")
console.log(navigator.languages); // Lista de idiomas preferidos
console.log(navigator.onLine); // true se há conexão com a internet
console.log(navigator.cookieEnabled); // true se cookies estão habilitados
console.log(navigator.platform); // Plataforma do sistema operacional

// Verificando suporte a recursos
if ("serviceWorker" in navigator) {
    console.log("Service Workers são suportados");
}

// Geolocalização
if ("geolocation" in navigator) {
    navigator.geolocation.getCurrentPosition(
        function(posicao) {
            console.log(`Latitude: ${posicao.coords.latitude}`);
            console.log(`Longitude: ${posicao.coords.longitude}`);
            console.log(`Precisão: ${posicao.coords.accuracy} metros`);
        },
        function(erro) {
            console.error(`Erro de geolocalização: ${erro.message}`);
        },
        { enableHighAccuracy: true, timeout: 5000 }
    );
}
```

15.5 O Objeto `location`

```
// Informações sobre a URL atual
console.log(location.href);      // URL completa: "https://exemplo.com/pagina?id=1#secao"
console.log(location.protocol); // "https:"
console.log(location.host);     // "exemplo.com"
console.log(location.hostname); // "exemplo.com" (sem porta)
console.log(location.port);     // "" (ou "8080" se especificado)
console.log(location.pathname); // "/pagina"
console.log(location.search);   // "?id=1"
console.log(location.hash);     // "#secao"
console.log(location.origin);   // "https://exemplo.com"

// Trabalhando com parâmetros de URL
const params = new URLSearchParams(location.search);
console.log(params.get("id"));  // "1"
params.set("pagina", "2");
console.log(params.toString()); // "id=1&pagina=2"

// Navegação programática
location.href = "https://outra-pagina.com"; // Redireciona (adiciona ao histórico)
location.replace("https://outra-pagina.com"); // Redireciona (NÃO adiciona ao histórico)
location.reload(); // Recarrega a página
location.reload(true); // Recarrega forçando busca no servidor
```


Capítulo 16: Web Storage e Cookies

16.1 localStorage: Armazenamento Persistente

O `localStorage` permite armazenar dados no navegador sem data de expiração. Os dados persistem mesmo após fechar o navegador e só são removidos explicitamente.

```
// Armazenando dados (apenas strings)
localStorage.setItem("tema", "escuro");
localStorage.setItem("idioma", "pt-BR");
localStorage.setItem("ultimoAcesso", new Date().toISOString());

// Lendo dados
const tema = localStorage.getItem("tema"); // "escuro"
const chaveInexistente = localStorage.getItem("xyz"); // null

// Verificando se um item existe
if (localStorage.getItem("tema") !== null) {
    aplicarTema(localStorage.getItem("tema"));
}

// Removendo um item específico
localStorage.removeItem("ultimoAcesso");

// Limpando todo o localStorage
localStorage.clear();

// Verificando o número de itens armazenados
console.log(localStorage.length);

// Acessando chaves por índice
for (let i = 0; i < localStorage.length; i++) {
    const chave = localStorage.key(i);
    const valor = localStorage.getItem(chave);
    console.log(`${chave}: ${valor}`);
}
```

16.2 Armazenando Objetos e Arrays

Como o Storage só aceita strings, use `JSON.stringify()` e `JSON.parse()` para trabalhar com dados complexos.

```
// Armazenando um objeto
const configuracoes = {
  tema: "escuro",
  tamanhoFonte: 16,
  notificacoes: true,
  idioma: "pt-BR"
};
localStorage.setItem("config", JSON.stringify(configuracoes));

// Recuperando o objeto
const configSalva = JSON.parse(localStorage.getItem("config"));
console.log(configSalva.tema); // "escuro"

// Armazenando um array
const favoritos = ["item1", "item2", "item3"];
localStorage.setItem("favoritos", JSON.stringify(favoritos));

const favoritosSalvos = JSON.parse(localStorage.getItem("favoritos"));
console.log(favoritosSalvos); // ["item1", "item2", "item3"]

// Padrão seguro: valor padrão se não existir
function carregarConfig() {
  const configPadrao = { tema: "claro", tamanhoFonte: 14 };
  const salva = localStorage.getItem("config");
  return salva ? JSON.parse(salva) : configPadrao;
}
```

16.3 `sessionStorage`

O `sessionStorage` funciona exatamente como o `localStorage`, mas os dados são apagados quando a sessão da aba termina (quando a aba é fechada).

```
// Salvando o estado de um formulário em múltiplas etapas
sessionStorage.setItem("etapa", "2");
sessionStorage.setItem("dadosEtapa1", JSON.stringify({ nome: "João", email: "joao@ex.com" }));

// Recuperando ao voltar para a etapa anterior
const etapaAtual = parseInt(sessionStorage.getItem("etapa")) || 1;
const dadosEtapa1 = JSON.parse(sessionStorage.getItem("dadosEtapa1"));
```

16.4 O Evento `storage`

O evento `storage` é disparado em outras abas/janelas do mesmo domínio quando o `localStorage` é modificado. É útil para sincronizar o estado entre múltiplas abas.

```
window.addEventListener("storage", function(e) {
    console.log(`Chave alterada: ${e.key}`);
    console.log(`Valor antigo: ${e.oldValue}`);
    console.log(`Novo valor: ${e.newValue}`);
    console.log(`URL da aba que alterou: ${e.url}`);

    // Sincronizando o tema entre abas
    if (e.key === "tema") {
        aplicarTema(e.newValue);
    }
});
```

16.5 Cookies com `document.cookie`

Cookies são mais antigos e complexos que o Web Storage, mas ainda são necessários para comunicação com o servidor (cookies são enviados automaticamente em requisições HTTP).

```
// Criando um cookie
document.cookie = "usuario=joao; expires=Fri, 31 Dec 2026 23:59:59 GMT; path=/";
document.cookie = "tema=escuro; max-age=86400; path=/"; // max-age em segundos

// Lendo cookies (retorna TODOS os cookies como uma string)
console.log(document.cookie); // "usuario=joao; tema=escuro"

// Função para ler um cookie específico
function getCookie(nome) {
  const cookies = document.cookie.split(";");
  for (const cookie of cookies) {
    const [chave, valor] = cookie.trim().split("=");
    if (chave === nome) return decodeURIComponent(valor);
  }
  return null;
}

// Deletando um cookie (definindo data de expiração no passado)
document.cookie = "usuario=; expires=Thu, 01 Jan 1970 00:00:00 GMT; path=/";
```

Capítulo 17: Drag and Drop API

17.1 Introdução ao Drag and Drop

A API de Drag and Drop nativa do HTML5 permite que os usuários arrastem elementos de um local para outro na página. Embora existam bibliotecas JavaScript para isso, a API nativa é poderosa e não requer dependências externas.

17.2 Tornando Elementos Arrastáveis

Para tornar um elemento arrastável, adicione o atributo `draggable="true"`.

```
<div class="item-kanban" id="tarefa-1" draggable="true">
  <h3>Implementar login</h3>
  <p>Criar sistema de autenticação JWT</p>
</div>
```

17.3 Eventos no Elemento Arrastado

Evento	Quando Dispara
<code>dragstart</code>	Quando o arrasto começa
<code>drag</code>	Continuamente durante o arrasto
<code>dragend</code>	Quando o arrasto termina (independente do resultado)

```
const itensArrastaveis = document.querySelectorAll("[draggable='true']");

itensArrastaveis.forEach(item => {
  item.addEventListener("dragstart", function(e) {
    // Armazena o ID do elemento sendo arrastado
    e.dataTransfer.setData("text/plain", this.id);
    e.dataTransfer.effectAllowed = "move"; // "copy", "move", "link", "all"

    // Adiciona classe visual
    this.classList.add("sendo-arrastado");

    // Define uma imagem customizada de arrasto (opcional)
    // const img = new Image();
    // img.src = "drag-icon.png";
    // e.dataTransfer.setDragImage(img, 0, 0);
  });

  item.addEventListener("dragend", function() {
    this.classList.remove("sendo-arrastado");
  });
});
```

17.4 Eventos na Zona de Soltura (Drop Zone)

Evento	Quando Dispara
<code>dragenter</code>	Quando o elemento arrastado entra na zona
<code>dragover</code>	Continuamente enquanto o elemento está sobre a zona
<code>dragleave</code>	Quando o elemento arrastado sai da zona sem soltar
<code>drop</code>	Quando o elemento é solto na zona

```
const zonasSoltura = document.querySelectorAll(".coluna-kanban");

zonasSoltura.forEach(zona => {
  zona.addEventListener("dragenter", function(e) {
    e.preventDefault();
    this.classList.add("zona-ativa");
  });

  zona.addEventListener("dragover", function(e) {
    e.preventDefault(); // OBRIGATÓRIO para permitir o drop
    e.dataTransfer.dropEffect = "move";
  });

  zona.addEventListener("dragleave", function(e) {
    // Verifica se saiu para um filho (evita falsos positivos)
    if (!this.contains(e.relatedTarget)) {
      this.classList.remove("zona-ativa");
    }
  });

  zona.addEventListener("drop", function(e) {
    e.preventDefault();
    this.classList.remove("zona-ativa");

    const idElemento = e.dataTransfer.getData("text/plain");
    const elementoArrastado = document.getElementById(idElemento);

    if (elementoArrastado) {
      this.appendChild(elementoArrastado);
    }
  });
});
```


Capítulo 18: MutationObserver e IntersectionObserver

18.1 MutationObserver: Monitorando Mudanças no DOM

O `MutationObserver` é uma API que permite observar mudanças na árvore do DOM. É muito mais eficiente do que verificar periodicamente o DOM com `setInterval`.

```

const alvo = document.getElementById("container-dinamico");

const observador = new MutationObserver(function(listaDeMutacoes, obs) {
  listaDeMutacoes.forEach(function(mutacao) {

    if (mutacao.type === "childList") {
      // Nós adicionados
      mutacao.addedNodes.forEach(no => {
        if (no.nodeType === Node.ELEMENT_NODE) {
          console.log("Elemento adicionado:", no.tagName);
        }
      });
      // Nós removidos
      mutacao.removedNodes.forEach(no => {
        if (no.nodeType === Node.ELEMENT_NODE) {
          console.log("Elemento removido:", no.tagName);
        }
      });
    }

    if (mutacao.type === "attributes") {
      console.log(`Atributo "${mutacao.attributeName}" modificado`);
      console.log(`Valor anterior: ${mutacao.oldValue}`);
    }

    if (mutacao.type === "characterData") {
      console.log("Conteúdo de texto modificado");
    }
  });
});

// Configurações do observador
const config = {
  childList: true,      // Observa adição/remoção de filhos
  attributes: true,     // Observa mudanças de atributos
  characterData: true,  // Observa mudanças de texto
  subtree: true,        // Observa toda a subárvore
  attributeOldValue: true, // Armazena o valor antigo dos atributos
};

```

```
characterDataOldValue: true // Armazena o valor antigo do texto
};

observador.observe(alvo, config);

// Para parar de observar
// observador.disconnect();

// Para obter mutações pendentes sem desconectar
// const mutacoesPendentes = observador.takeRecords();
```

18.2 IntersectionObserver: Detectando Visibilidade

O `IntersectionObserver` detecta quando um elemento entra ou sai da área visível (viewport) ou de um elemento pai com scroll. É a base para implementar lazy loading, infinite scroll e animações ao rolar.

```
// Configuração básica
const opcoes = {
  root: null,          // null = viewport do navegador
  rootMargin: "0px",  // Margem ao redor do root (como CSS margin)
  threshold: 0.5       // 50% do elemento deve estar visível para disparar
  // threshold: [0, 0.25, 0.5, 0.75, 1] // Múltiplos thresholds
};

const observador = new IntersectionObserver(function(entradas) {
  entradas.forEach(entrada => {
    console.log(`Elemento: ${entrada.target.id}`);
    console.log(`Está visível: ${entrada.isIntersecting}`);
    console.log(`Proporção visível: ${entrada.intersectionRatio}`);
    console.log(`Retângulo de interseção:`, entrada.intersectionRect);
  });
}, opcoes);

// Observando elementos
document.querySelectorAll(".secao").forEach(secao => {
  observador.observe(secao);
});
```

18.3 Lazy Loading de Imagens

```
const imagensLazy = document.querySelectorAll("img[data-src]");

const lazyObserver = new IntersectionObserver(function(entradas, obs) {
  entradas.forEach(entrada => {
    if (entrada.isIntersecting) {
      const img = entrada.target;

      // Carrega a imagem real
      img.src = img.dataset.src;
      img.classList.add("carregada");

      // Para de observar esta imagem
      obs.unobserve(img);
    }
  });
}, { rootMargin: "200px" }); // Começa a carregar 200px antes de aparecer

imagensLazy.forEach(img => lazyObserver.observe(img));
```

18.4 Animações ao Rolar (Scroll Animations)

```
const elementosAnimados = document.querySelectorAll(".animar-ao-aparecer");

const animacaoObserver = new IntersectionObserver(function(entradas) {
  entradas.forEach(entrada => {
    if (entrada.isIntersecting) {
      entrada.target.classList.add("visivel");
    } else {
      entrada.target.classList.remove("visivel"); // Remove ao sair
    }
  });
}, { threshold: 0.15 });

elementosAnimados.forEach(el => animacaoObserver.observe(el));
```

18.5 ResizeObserver

O `ResizeObserver` monitora mudanças no tamanho de um elemento específico (não da janela inteira).

```
const caixaRedimensionavel = document.querySelector(".caixa");

const resizeObs = new ResizeObserver(function(entradas) {
  entradas.forEach(entrada => {
    const { width, height } = entrada.contentRect;
    console.log(`Nova largura: ${width}px, Nova altura: ${height}px`);

    // Adaptar o conteúdo baseado no tamanho do contêiner
    if (width < 400) {
      entrada.target.classList.add("modo-compacto");
    } else {
      entrada.target.classList.remove("modo-compacto");
    }
  });
});

resizeObs.observe(caixaRedimensionavel);
```

Capítulo 19: Canvas e SVG no DOM

19.1 O Elemento `<canvas>`

O elemento `<canvas>` fornece uma área de desenho via JavaScript. É usado para gráficos, jogos, visualizações de dados e animações complexas.

```
<canvas id="meuCanvas" width="600" height="400"></canvas>
```



```
const canvas = document.getElementById("meuCanvas");
const ctx = canvas.getContext("2d");

// Retângulos
ctx.fillStyle = "#3498db";
ctx.fillRect(10, 10, 100, 80);    // Retângulo preenchido

ctx.strokeStyle = "#e74c3c";
ctx.lineWidth = 3;
ctx.strokeRect(130, 10, 100, 80); // Retângulo com borda

ctx.clearRect(20, 20, 30, 30);    // Limpa uma área

// Caminhos (paths)
ctx.beginPath();
ctx.moveTo(250, 10);
ctx.lineTo(350, 90);
ctx.lineTo(250, 90);
ctx.closePath();
ctx.fillStyle = "#2ecc71";
ctx.fill();
ctx.stroke();

// Círculos e arcos
ctx.beginPath();
ctx.arc(450, 50, 40, 0, Math.PI * 2); // x, y, raio, ângulo inicial, ângulo final
ctx.fillStyle = "#9b59b6";
ctx.fill();

// Texto
ctx.font = "bold 24px Arial";
ctx.fillStyle = "#333";
ctx.fillText("Olá Canvas!", 10, 150);
ctx.strokeText("Texto com borda", 10, 190);

// Gradientes
const gradiente = ctx.createLinearGradient(0, 200, 600, 200);
gradiente.addColorStop(0, "#3498db");
```

```
gradiente.addColorStop(1, "#e74c3c");  
ctx.fillStyle = gradiente;  
ctx.fillRect(0, 200, 600, 50);
```

19.2 Imagens no Canvas

```
const img = new Image();  
img.onload = function() {  
    ctx.drawImage(img, 0, 0); // Posição original  
    ctx.drawImage(img, 0, 0, 200, 150); // Com dimensões  
    ctx.drawImage(img, 50, 50, 100, 100, 300, 0, 200, 200); // Recorte + posição  
};  
img.src = "foto.jpg";
```

19.3 Manipulando SVG no DOM

SVG (Scalable Vector Graphics) é um formato de imagem vetorial baseado em XML. Elementos SVG inline em HTML podem ser manipulados diretamente com JavaScript, assim como qualquer outro elemento DOM.

```
<svg id="meu-svg" width="200" height="200" xmlns="http://www.w3.org/2000/svg">  
    <circle id="circulo" cx="100" cy="100" r="50" fill="blue"/>  
    <rect id="retangulo" x="10" y="10" width="80" height="60" fill="red"/>  
    <text id="texto-svg" x="100" y="180" text-anchor="middle">SVG</text>  
</svg>
```

```
const circulo = document.getElementById("circulo");

// Manipulando atributos SVG
circulo.setAttribute("fill", "#e74c3c");
circulo.setAttribute("r", "70");
circulo.setAttribute("cx", "100");

// Animação de um elemento SVG
let angulo = 0;
setInterval(() => {
    angulo += 2;
    const x = 100 + 60 * Math.cos(angulo * Math.PI / 180);
    const y = 100 + 60 * Math.sin(angulo * Math.PI / 180);
    circulo.setAttribute("cx", x);
    circulo.setAttribute("cy", y);
}, 16);

// Criando elementos SVG via JavaScript
const svgNS = "http://www.w3.org/2000/svg";
const novoCirculo = document.createElementNS(svgNS, "circle");
novoCirculo.setAttribute("cx", "150");
novoCirculo.setAttribute("cy", "50");
novoCirculo.setAttribute("r", "30");
novoCirculo.setAttribute("fill", "green");
document.getElementById("meu-svg").appendChild(novoCirculo);
```

Capítulo 20: Projetos Práticos Completos

Projeto 1: Todo List Avançado com LocalStorage

Este projeto implementa uma lista de tarefas completa com criação, leitura, atualização e exclusão (CRUD), persistência de dados e filtragem.

```

<!DOCTYPE html>
<html lang="pt-BR">
<head>
  <meta charset="UTF-8">
  <title>Todo List</title>
  <style>
    * { box-sizing: border-box; margin: 0; padding: 0; }
    body { font-family: Arial, sans-serif; max-width: 600px; margin: 40px auto; padding: 20px; }
    h1 { text-align: center; margin-bottom: 20px; color: #333; }
    .form-adicionar { display: flex; gap: 10px; margin-bottom: 20px; }
    .form-adicionar input { flex: 1; padding: 10px; border: 1px solid #ddd; border-radius: 4px; }
    .form-adicionar button { padding: 10px 20px; background: #3498db; color: white; border: none; border-radius: 4px; }
    .filtros { display: flex; gap: 10px; margin-bottom: 20px; }
    .filtros button { padding: 8px 16px; border: 1px solid #ddd; border-radius: 4px; cursor: pointer; }
    .filtros button.ativo { background: #3498db; color: white; border-color: #3498db; }
    .tarefa { display: flex; align-items: center; gap: 10px; padding: 12px; border: 1px solid #ddd; margin-bottom: 10px; }
    .tarefa.concluida .texto-tarefa { text-decoration: line-through; color: #999; }
    .texto-tarefa { flex: 1; font-size: 16px; }
    .btn-excluir { background: #e74c3c; color: white; border: none; border-radius: 4px; padding: 5px 10px; }
    .contador { text-align: center; color: #666; margin-top: 10px; }
  </style>
</head>
<body>
  <h1>Minhas Tarefas</h1>
  <div class="form-adicionar">
    <input type="text" id="input-tarefa" placeholder="Adicionar nova tarefa...">
    <button id="btn-adicionar">Adicionar</button>
  </div>
  <div class="filtros">
    <button class="ativo" data-filtro="todas">Todas</button>
    <button data-filtro="ativas">Ativas</button>
    <button data-filtro="concluidas">Concluídas</button>
  </div>
  <div id="lista-tarefas"></div>
  <p class="contador" id="contador"></p>
  <script src="todo.js"></script>
</body>
</html>

```

```
// todo.js

let tarefas = JSON.parse(localStorage.getItem("tarefas")) || [];
let filtroAtual = "todas";
let proximoId = parseInt(localStorage.getItem("proximoId")) || 1;

function salvar() {
  localStorage.setItem("tarefas", JSON.stringify(tarefas));
  localStorage.setItem("proximoId", proximoId);
}

function adicionarTarefa(texto) {
  if (!texto.trim()) return;
  tarefas.push({ id: proximoId++, texto: texto.trim(), concluida: false });
  salvar();
  renderizar();
}

function alternarTarefa(id) {
  const tarefa = tarefas.find(t => t.id === id);
  if (tarefa) { tarefa.concluida = !tarefa.concluida; salvar(); renderizar(); }
}

function excluirTarefa(id) {
  tarefas = tarefas.filter(t => t.id !== id);
  salvar();
  renderizar();
}

function renderizar() {
  const lista = document.getElementById("lista-tarefas");
  const tarefasFiltradas = tarefas.filter(t => {
    if (filtroAtual === "ativas") return !t.concluida;
    if (filtroAtual === "concluidas") return t.concluida;
    return true;
  });

  lista.innerHTML = "";
  tarefasFiltradas.forEach(tarefa => {
```

```

    const div = document.createElement("div");
    div.className = `tarefa${tarefa.concluida ? " concluida" : ""}`;
    div.innerHTML = `
        <input type="checkbox" ${tarefa.concluida ? "checked" : ""}>
        <span class="texto-tarefa">${tarefa.texto}</span>
        <button class="btn-excluir">Excluir</button>
    `;
    div.querySelector("input").addEventListener("change", () => alternarTarefa(tarefa.id));
    div.querySelector("button").addEventListener("click", () => excluirTarefa(tarefa.id));
    lista.appendChild(div);
  });

  const ativas = tarefas.filter(t => !t.concluida).length;
  document.getElementById("contador").textContent = `${ativas} tarefa(s) pendente(s)`;
}

document.getElementById("btn-adicionar").addEventListener("click", () => {
  const input = document.getElementById("input-tarefa");
  adicionarTarefa(input.value);
  input.value = "";
  input.focus();
});

document.getElementById("input-tarefa").addEventListener("keypress", e => {
  if (e.key === "Enter") document.getElementById("btn-adicionar").click();
});

document.querySelector(".filtros").addEventListener("click", e => {
  if (e.target.dataset.filtro) {
    document.querySelectorAll(".filtros button").forEach(b => b.classList.remove("ativo"));
    e.target.classList.add("ativo");
    filtroAtual = e.target.dataset.filtro;
    renderizar();
  }
});

renderizar();

```

Projeto 2: Validador de Formulário Dinâmico


```
// Configuração das regras de validação
const regras = {
  nome: [
    { teste: v => v.trim().length > 0, mensagem: "O nome é obrigatório." },
    { teste: v => v.trim().length >= 3, mensagem: "O nome deve ter pelo menos 3 caracteres." }
  ],
  email: [
    { teste: v => v.trim().length > 0, mensagem: "O e-mail é obrigatório." },
    { teste: v => /^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(v), mensagem: "Insira um e-mail válido." }
  ],
  senha: [
    { teste: v => v.length >= 8, mensagem: "A senha deve ter pelo menos 8 caracteres." },
    { teste: v => /[A-Z]/.test(v), mensagem: "A senha deve conter pelo menos uma letra maiúscula." },
    { teste: v => /[0-9]/.test(v), mensagem: "A senha deve conter pelo menos um número." }
  ]
};

function validarCampo(campo) {
  const nome = campo.name;
  const valor = campo.value;
  const erros = regras[nome] || [];

  const mensagemErro = campo.parentElement.querySelector(".erro");

  for (const regra of erros) {
    if (!regra.teste(valor)) {
      campo.classList.add("invalido");
      campo.classList.remove("valido");
      if (mensagemErro) mensagemErro.textContent = regra.mensagem;
      return false;
    }
  }

  campo.classList.remove("invalido");
  campo.classList.add("valido");
  if (mensagemErro) mensagemErro.textContent = "";
  return true;
}
```

```
const form = document.querySelector("#form-registro");

// Valida ao sair do campo
form.querySelectorAll("input").forEach(input => {
  input.addEventListener("blur", () => validarCampo(input));
  input.addEventListener("input", () => {
    if (input.classList.contains("invalido")) validarCampo(input);
  });
});

form.addEventListener("submit", function(e) {
  e.preventDefault();
  const campos = form.querySelectorAll("input[name]");
  let formularioValido = true;

  campos.forEach(campo => {
    if (!validarCampo(campo)) formularioValido = false;
  });

  if (formularioValido) {
    console.log("Formulário válido! Enviando...");
  }
});
```

Projeto 3: Tabela com Ordenação e Filtro Dinâmico

```

const dados = [
  { id: 1, nome: "Alice", departamento: "TI", salario: 8500 },
  { id: 2, nome: "Bruno", departamento: "RH", salario: 6200 },
  { id: 3, nome: "Carla", departamento: "TI", salario: 9100 },
  { id: 4, nome: "Diego", departamento: "Financeiro", salario: 7800 },
  { id: 5, nome: "Elena", departamento: "RH", salario: 6800 }
];

let dadosFiltrados = [...dados];
let colunaOrdenacao = null;
let direcaoOrdenacao = "asc";

function renderizarTabela() {
  const tbody = document.querySelector("#tabela tbody");
  tbody.innerHTML = "";

  dadosFiltrados.forEach(item => {
    const tr = document.createElement("tr");
    tr.innerHTML = `
      <td>${item.id}</td>
      <td>${item.nome}</td>
      <td>${item.departamento}</td>
      <td>R$ ${item.salario.toLocaleString("pt-BR")}</td>
    `;
    tbody.appendChild(tr);
  });

  document.querySelector("#total").textContent = `Total: ${dadosFiltrados.length} registros`;
}

function filtrar(termoBusca) {
  const termo = termoBusca.toLowerCase();
  dadosFiltrados = dados.filter(item =>
    item.nome.toLowerCase().includes(termo) ||
    item.departamento.toLowerCase().includes(termo)
  );
  renderizarTabela();
}

```

```
function ordenar(coluna) {
  if (colunaOrdenacao === coluna) {
    direcaoOrdenacao = direcaoOrdenacao === "asc" ? "desc" : "asc";
  } else {
    colunaOrdenacao = coluna;
    direcaoOrdenacao = "asc";
  }

  dadosFiltrados.sort((a, b) => {
    const valA = a[coluna];
    const valB = b[coluna];
    const comparacao = typeof valA === "string" ? valA.localeCompare(valB) : valA - valB;
    return direcaoOrdenacao === "asc" ? comparacao : -comparacao;
  });

  renderizarTabela();
}

document.querySelector("#busca").addEventListener("input", e => filtrar(e.target.value));
document.querySelectorAll("th[data-coluna]").forEach(th => {
  th.style.cursor = "pointer";
  th.addEventListener("click", () => ordenar(th.dataset.coluna));
});

renderizarTabela();
```

Apêndice: Tabelas de Referência Rápida

A.1 Tabela Completa de Eventos do DOM

Categoria	Evento	Descrição	Faz Bubble?
Mouse	<code>click</code>	Clique simples	Sim
Mouse	<code>dblclick</code>	Clique duplo	Sim
Mouse	<code>mousedown</code>	Botão pressionado	Sim
Mouse	<code>mouseup</code>	Botão solto	Sim
Mouse	<code>mousemove</code>	Mouse em movimento	Sim
Mouse	<code>mouseenter</code>	Mouse entra no elemento	Não
Mouse	<code>mouseleave</code>	Mouse sai do elemento	Não
Mouse	<code>mouseover</code>	Mouse entra (com bubble)	Sim
Mouse	<code>mouseout</code>	Mouse sai (com bubble)	Sim
Mouse	<code>contextmenu</code>	Botão direito	Sim
Mouse	<code>wheel</code>	Roda do mouse	Sim
Teclado	<code>keydown</code>	Tecla pressionada	Sim
Teclado	<code>keyup</code>	Tecla solta	Sim
Formulário	<code>submit</code>	Formulário enviado	Sim
Formulário	<code>input</code>	Valor alterado	Sim
Formulário	<code>change</code>	Valor alterado + blur	Sim
Formulário	<code>focus</code>	Elemento focado	Não
Formulário	<code>blur</code>	Elemento desfocado	Não
Formulário	<code>focusin</code>	Elemento focado	Sim
Formulário	<code>focusout</code>	Elemento desfocado	Sim
Formulário	<code>reset</code>	Formulário resetado	Sim

Categoria	Evento	Descrição	Faz Bubble?
Formulário	<code>invalid</code>	Campo inválido	Sim
Janela	<code>load</code>	Página carregada	Não
Janela	<code>DOMContentLoaded</code>	DOM pronto	Não
Janela	<code>resize</code>	Janela redimensionada	Não
Janela	<code>scroll</code>	Página rolada	Não
Janela	<code>beforeunload</code>	Antes de sair	Não
Janela	<code>unload</code>	Ao sair	Não
Janela	<code>hashchange</code>	Hash da URL mudou	Não
Janela	<code>popstate</code>	Histórico mudou	Não
Janela	<code>visibilitychange</code>	Visibilidade da aba	Não
Janela	<code>online</code>	Conexão restabelecida	Não
Janela	<code>offline</code>	Conexão perdida	Não
Janela	<code>storage</code>	localStorage alterado	Não
Drag & Drop	<code>dragstart</code>	Arrasto iniciado	Sim
Drag & Drop	<code>drag</code>	Durante o arrasto	Sim
Drag & Drop	<code>dragend</code>	Arrasto terminado	Sim
Drag & Drop	<code>dragenter</code>	Entrou na zona	Sim
Drag & Drop	<code>dragover</code>	Sobre a zona	Sim
Drag & Drop	<code>dragleave</code>	Saiu da zona	Sim
Drag & Drop	<code>drop</code>	Solto na zona	Sim
Clipboard	<code>copy</code>	Copiar	Sim

Categoria	Evento	Descrição	Faz Bubble?
Clipboard	<code>cut</code>	Recortar	Sim
Clipboard	<code>paste</code>	Colar	Sim
Mídia	<code>play</code>	Reprodução iniciada	Não
Mídia	<code>pause</code>	Reprodução pausada	Não
Mídia	<code>ended</code>	Reprodução finalizada	Não
Mídia	<code>timeupdate</code>	Posição atualizada	Não
Mídia	<code>volumechange</code>	Volume alterado	Não
Mídia	<code>loadedmetadata</code>	Metadados carregados	Não
Mídia	<code>canplay</code>	Pode reproduzir	Não
Mídia	<code>error</code>	Erro de mídia	Não
Pointer	<code>pointerdown</code>	Ponteiro pressionado	Sim
Pointer	<code>pointerup</code>	Ponteiro solto	Sim
Pointer	<code>pointermove</code>	Ponteiro em movimento	Sim
Pointer	<code>pointerenter</code>	Ponteiro entrou	Não
Pointer	<code>pointerleave</code>	Ponteiro saiu	Não
Pointer	<code>pointercancel</code>	Ponteiro cancelado	Sim
Touch	<code>touchstart</code>	Toque iniciado	Sim
Touch	<code>touchend</code>	Toque finalizado	Sim
Touch	<code>touchmove</code>	Toque em movimento	Sim
Touch	<code>touchcancel</code>	Toque cancelado	Sim
Animação CSS	<code>animationstart</code>	Animação iniciada	Sim

Categoria	Evento	Descrição	Faz Bubble?
Animação CSS	<code>animationend</code>	Animação finalizada	Sim
Animação CSS	<code>animationiteration</code>	Iteração da animação	Sim
Transição CSS	<code>transitionstart</code>	Transição iniciada	Sim
Transição CSS	<code>transitionend</code>	Transição finalizada	Sim

A.2 Tabela de Métodos de Seleção

Método	Retorna	Vivo?	Chamado em
<code>getElementById(id)</code>	Element ou null	N/A	document
<code>getElementsByClassName(classe)</code>	HTMLCollection	Sim	document, Element
<code>getElementsByTagName(tag)</code>	HTMLCollection	Sim	document, Element
<code>querySelector(seletor)</code>	Element ou null	N/A	document, Element
<code>querySelectorAll(seletor)</code>	NodeList	Não	document, Element
<code>getElementsByName(nome)</code>	NodeList	Sim	document

A.3 Tabela de Métodos de Inserção

Método	Posição de Inserção	Aceita
<code>appendChild(no)</code>	Último filho	Nó
<code>prepend(...nos)</code>	Primeiro filho	Nós e strings
<code>append(...nos)</code>	Último filho	Nós e strings
<code>insertBefore(novo, ref)</code>	Antes do nó de referência	Nó
<code>insertAdjacentHTML("beforebegin", html)</code>	Antes do elemento	String HTML
<code>insertAdjacentHTML("afterbegin", html)</code>	Primeiro filho	String HTML
<code>insertAdjacentHTML("beforeend", html)</code>	Último filho	String HTML
<code>insertAdjacentHTML("afterend", html)</code>	Após o elemento	String HTML
<code>replaceChild(novo, antigo)</code>	Substitui filho	Nó
<code>replaceWith(...nos)</code>	Substitui o próprio elemento	Nós e strings

A.4 Tabela de Propriedades de Navegação

Propriedade	Retorna	Inclui Nós de Texto?
<code>parentNode</code>	Nó pai	Sim
<code>parentElement</code>	Elemento pai	Não
<code>childNodes</code>	NodeList de filhos	Sim
<code>children</code>	HTMLCollection de filhos	Não
<code>firstChild</code>	Primeiro filho	Sim
<code>lastChild</code>	Último filho	Sim
<code>firstElementChild</code>	Primeiro filho elemento	Não
<code>lastElementChild</code>	Último filho elemento	Não
<code>nextSibling</code>	Próximo irmão	Sim
<code>previousSibling</code>	Irmão anterior	Sim
<code>nextElementSibling</code>	Próximo irmão elemento	Não
<code>previousElementSibling</code>	Irmão anterior elemento	Não
<code>childElementCount</code>	Número de filhos elementos	N/A

Fim da Apostila — JavaScript DOM: O Guia Definitivo

Manus AI — 2026

Capítulo 21: Manipulação Avançada do DOM

21.1 O Método `matches()`

O método `matches(seletor)` verifica se um elemento corresponde a um seletor CSS específico, retornando `true` ou `false`. É extremamente útil em conjunto com a delegação de eventos.

```
const link = document.querySelector("a");

// Verifica se o link tem a classe "externo"
if (link.matches(".externo")) {
    link.setAttribute("target", "_blank");
    link.setAttribute("rel", "noopener noreferrer");
}

// Uso em delegação de eventos
document.addEventListener("click", function(e) {
    if (e.target.matches("button.btn-primario")) {
        e.target.classList.add("clicado");
    }

    if (e.target.matches("a[href^='http']")) {
        // Link externo clicado
        registrarClique(e.target.href);
    }

    if (e.target.matches("input[type='checkbox']")) {
        // Qualquer checkbox clicado
        console.log(`Checkbox ${e.target.name}: ${e.target.checked}`);
    }
});
```

21.2 O Método `getBoundingClientRect()`

Este método retorna um objeto `DOMRect` com as dimensões e posição de um elemento em relação à viewport.

```

const elemento = document.querySelector(".meu-elemento");
const rect = elemento.getBoundingClientRect();

console.log(`Top: ${rect.top}px`);      // Distância do topo da viewport
console.log(`Left: ${rect.left}px`);    // Distância da esquerda da viewport
console.log(`Bottom: ${rect.bottom}px`); // Distância do topo até a borda inferior
console.log(`Right: ${rect.right}px`);  // Distância da esquerda até a borda direita
console.log(`Width: ${rect.width}px`);  // Largura do elemento
console.log(`Height: ${rect.height}px`); // Altura do elemento
console.log(`X: ${rect.x}px`);          // Equivalente a left
console.log(`Y: ${rect.y}px`);          // Equivalente a top

// Verificando se um elemento está visível na viewport
function estaVisivel(elemento) {
    const rect = elemento.getBoundingClientRect();
    return (
        rect.top >= 0 &&
        rect.left >= 0 &&
        rect.bottom <= window.innerHeight &&
        rect.right <= window.innerWidth
    );
}

// Posicionando um elemento em relação a outro
function posicionarAbaixo(alvo, referencia) {
    const rect = referencia.getBoundingClientRect();
    alvo.style.position = "fixed";
    alvo.style.top = `${rect.bottom + 5}px`;
    alvo.style.left = `${rect.left}px`;
    alvo.style.width = `${rect.width}px`;
}

```

21.3 Dimensões e Posicionamento de Elementos

O DOM oferece várias propriedades para obter dimensões de elementos:

Propriedade	Inclui	Descrição
<code>clientWidth</code> / <code>clientHeight</code>	Conteúdo + Padding	Tamanho interno visível
<code>offsetWidth</code> / <code>offsetHeight</code>	Conteúdo + Padding + Border	Tamanho total com borda
<code>scrollWidth</code> / <code>scrollHeight</code>	Conteúdo total (incluindo overflow)	Tamanho total do conteúdo
<code>offsetTop</code> / <code>offsetLeft</code>	—	Posição relativa ao <code>offsetParent</code>
<code>scrollTop</code> / <code>scrollLeft</code>	—	Quantidade de scroll do elemento


```
const div = document.querySelector(".caixa-scroll");

// Dimensões
console.log(`clientWidth: ${div.clientWidth}px`);
console.log(`offsetWidth: ${div.offsetWidth}px`);
console.log(`scrollWidth: ${div.scrollWidth}px`);

// Posição
console.log(`offsetTop: ${div.offsetTop}px`);
console.log(`offsetLeft: ${div.offsetLeft}px`);

// Scroll
console.log(`scrollTop: ${div.scrollTop}px`);

// Rolar para o final do elemento
div.scrollTop = div.scrollHeight;

// Verificar se chegou ao final do scroll
div.addEventListener("scroll", function() {
    const chegouAoFinal = this.scrollTop + this.clientHeight >= this.scrollHeight - 5;
    if (chegouAoFinal) {
        console.log("Chegou ao final do conteúdo!");
    }
});
```

21.4 O Método `scrollIntoView()`

```
const secao = document.querySelector("#secao-contato");

// Rola suavemente até o elemento
secao.scrollIntoView({ behavior: "smooth", block: "start" });

// Opções:
// behavior: "smooth" | "instant" | "auto"
// block: "start" | "center" | "end" | "nearest" (alinhamento vertical)
// inline: "start" | "center" | "end" | "nearest" (alinhamento horizontal)

// Navegação por âncoras customizada
document.querySelectorAll("a[href^='#']").forEach(link => {
  link.addEventListener("click", function(e) {
    e.preventDefault();
    const alvo = document.querySelector(this.getAttribute("href"));
    if (alvo) {
      alvo.scrollIntoView({ behavior: "smooth", block: "start" });
      // Atualiza a URL sem recarregar
      history.pushState(null, null, this.getAttribute("href"));
    }
  });
});
```

21.5 `focus()`, `blur()` e `click()` Programáticos

```
// Focar um elemento programaticamente
const input = document.querySelector("#campo-busca");
input.focus();
input.focus({ preventScroll: true }); // Foca sem rolar a página

// Desfoca o elemento atualmente focado
document.activeElement.blur();

// Simular um clique
const botao = document.querySelector("#btn-confirmar");
botao.click();

// Selecionar todo o texto de um input
const inputTexto = document.querySelector("input[type='text']");
inputTexto.select();

// Selecionar um intervalo de texto
inputTexto.setSelectionRange(0, 5); // Seleciona os primeiros 5 caracteres
```

Capítulo 22: Formulários Avançados

22.1 A Interface `HTMLFormElement`

O elemento `<form>` possui propriedades e métodos especiais no DOM:

```
const form = document.querySelector("#meu-form");

// Acessando todos os campos do formulário
console.log(form.elements);           // HTMLFormControlsCollection
console.log(form.elements.length);    // Número de campos
console.log(form.elements["email"]);  // Campo com name="email"
console.log(form.elements[0]);        // Primeiro campo

// Propriedades do formulário
console.log(form.action);              // URL de envio
console.log(form.method);              // "get" ou "post"
console.log(form enctype);             // Tipo de codificação

// Métodos
form.reset();                          // Reseta todos os campos para os valores padrão
form.submit();                         // Envia o formulário (sem disparar o evento submit)
form.requestSubmit();                 // Envia o formulário (dispara o evento submit)
```

22.2 Trabalhando com `<select>` Múltiplo

```
const selectMultiplo = document.querySelector("select[multiple]);

// Obtendo todos os valores selecionados
const valoresSelecionados = Array.from(selectMultiplo.selectedOptions)
  .map(option => option.value);

console.log(valoresSelecionados); // ["opcao1", "opcao3"]

// Selecionando opções programaticamente
Array.from(selectMultiplo.options).forEach(option => {
  if (["opcao1", "opcao2"].includes(option.value)) {
    option.selected = true;
  }
});

// Adicionando opções dinamicamente
const novaOpcao = new Option("Nova Opção", "nova-opcao");
selectMultiplo.add(novaOpcao);

// Removendo uma opção
selectMultiplo.remove(0); // Remove a primeira opção
```

22.3 Upload de Arquivos com `<input type="file">`

```
const inputArquivo = document.querySelector("input[type='file']");

inputArquivo.addEventListener("change", function() {
  const arquivos = this.files; // FileList

  Array.from(arquivos).forEach(arquivo => {
    console.log(`Nome: ${arquivo.name}`);
    console.log(`Tamanho: ${((arquivo.size / 1024).toFixed(2))} KB`);
    console.log(`Tipo: ${arquivo.type}`);
    console.log(`Última modificação: ${new Date(arquivo.lastModified)}`);

    // Lendo o conteúdo do arquivo
    const reader = new FileReader();

    reader.onload = function(e) {
      if (arquivo.type.startsWith("image/")) {
        // Pré-visualização de imagem
        const img = document.createElement("img");
        img.src = e.target.result;
        img.style.maxWidth = "200px";
        document.querySelector("#preview").appendChild(img);
      } else if (arquivo.type === "text/plain") {
        // Exibir conteúdo de texto
        document.querySelector("#conteudo-arquivo").textContent = e.target.result;
      }
    };

    if (arquivo.type.startsWith("image/")) {
      reader.readAsDataURL(arquivo);
    } else {
      reader.readAsText(arquivo, "UTF-8");
    }
  });
});
```

Capítulo 23: Animações com JavaScript e CSS

23.1 Animações com `requestAnimationFrame`

```
// Animação de fade in
function fadeIn(elemento, duracao = 500) {
  elemento.style.opacity = 0;
  elemento.style.display = "block";

  let inicio = null;

  function passo(timestamp) {
    if (!inicio) inicio = timestamp;
    const progresso = timestamp - inicio;
    const opacidade = Math.min(progresso / duracao, 1);

    elemento.style.opacity = opacidade;

    if (progresso < duracao) {
      requestAnimationFrame(passo);
    }
  }

  requestAnimationFrame(passo);
}

// Animação de slide down
function slideDown(elemento, duracao = 400) {
  elemento.style.display = "block";
  const alturaTotal = elemento.scrollHeight;
  elemento.style.height = "0";
  elemento.style.overflow = "hidden";

  let inicio = null;

  function passo(timestamp) {
    if (!inicio) inicio = timestamp;
    const progresso = timestamp - inicio;
    const percentual = Math.min(progresso / duracao, 1);

    // Easing: ease-out
    const eased = 1 - Math.pow(1 - percentual, 3);
  }
}
```



```

    elemento.style.height = `${alturaTotal * eased}px`;

    if (progresso < duracao) {
        requestAnimationFrame(passo);
    } else {
        elemento.style.height = "";
        elemento.style.overflow = "";
    }
}

requestAnimationFrame(passo);
}

// Animação de movimento
function moverPara(elemento, destinoX, destinoY, duracao = 600) {
    const inicioX = elemento.offsetLeft;
    const inicioY = elemento.offsetTop;
    let inicio = null;

    function passo(timestamp) {
        if (!inicio) inicio = timestamp;
        const progresso = timestamp - inicio;
        const t = Math.min(progresso / duracao, 1);

        // Easing: ease-in-out
        const eased = t < 0.5 ? 2 * t * t : -1 + (4 - 2 * t) * t;

        elemento.style.left = `${inicioX + (destinoX - inicioX) * eased}px`;
        elemento.style.top = `${inicioY + (destinoY - inicioY) * eased}px`;

        if (progresso < duracao) {
            requestAnimationFrame(passo);
        }
    }

    requestAnimationFrame(passo);
}

```

23.2 Eventos de Animação e Transição CSS

```
const elemento = document.querySelector(".caixa-animada");

// Eventos de Animação CSS
elemento.addEventListener("animationstart", function(e) {
    console.log(`Animação "${e.animationName}" iniciada`);
});

elemento.addEventListener("animationend", function(e) {
    console.log(`Animação "${e.animationName}" finalizada`);
    this.classList.remove("animando"); // Remove a classe após a animação
});

elemento.addEventListener("animationiteration", function(e) {
    console.log(`Iteração da animação "${e.animationName}"`);
});

// Eventos de Transição CSS
elemento.addEventListener("transitionstart", function(e) {
    console.log(`Transição da propriedade "${e.propertyName}" iniciada`);
});

elemento.addEventListener("transitionend", function(e) {
    console.log(`Transição da propriedade "${e.propertyName}" finalizada`);
    console.log(`Duração: ${e.elapsedTime}s`);

    // Encadear ações após a transição
    if (e.propertyName === "opacity" && this.style.opacity === "0") {
        this.style.display = "none";
    }
});
```

23.3 Web Animations API

A Web Animations API permite criar animações complexas diretamente via JavaScript, com controle total sobre reprodução, pausa, velocidade e direção.

```

const elemento = document.querySelector(".caixa");

// Criando uma animação
const animacao = elemento.animate(
  // Keyframes
  [
    { transform: "translateX(0px)", opacity: 1 },
    { transform: "translateX(200px)", opacity: 0.5, offset: 0.7 },
    { transform: "translateX(300px)", opacity: 0 }
  ],
  // Opções
  {
    duration: 1000,      // Duração em ms
    easing: "ease-in-out",
    fill: "forwards",    // "none" | "forwards" | "backwards" | "both"
    iterations: 1,       // Número de repetições (Infinity para infinito)
    direction: "normal", // "normal" | "reverse" | "alternate"
    delay: 500           // Atraso em ms
  }
);

// Controlando a animação
animacao.pause();
animacao.play();
animacao.reverse();
animacao.cancel();
animacao.finish();

// Propriedades
console.log(animacao.playState); // "running" | "paused" | "finished"
console.log(animacao.currentTime); // Posição atual em ms
animacao.playbackRate = 2;        // Velocidade (2 = dobro da velocidade)

// Evento de conclusão
animacao.onfinish = function() {
  console.log("Animação concluída!");
};

```

```
// Obtendo todas as animações de um elemento  
const todasAnimacoes = elemento.getAnimations();
```

Capítulo 24: APIs Modernas do DOM

24.1 A API de Seleção de Texto (`Selection`)

```

// Obtendo o texto selecionado pelo usuário
document.addEventListener("mouseup", function() {
    const selecao = window.getSelection();

    if (selecao.toString().trim() !== "") {
        console.log(`Texto selecionado: "${selecao.toString()}`);
        console.log(`Número de intervalos: ${selecao.rangeCount}`);

        const intervalo = selecao.getRangeAt(0);
        console.log(`Elemento inicial: ${intervalo.startContainer.nodeName}`);
        console.log(`Offset inicial: ${intervalo.startOffset}`);

        // Obtendo o retângulo da seleção
        const rect = intervalo.getBoundingClientRect();
        mostrarMenuContexto(rect.left, rect.top);
    }
});

// Selecionando texto programaticamente
function selecionarTexto(elemento) {
    const intervalo = document.createRange();
    intervalo.selectNodeContents(elemento);

    const selecao = window.getSelection();
    selecao.removeAllRanges();
    selecao.addRange(intervalo);
}

// Limpando a seleção
window.getSelection().removeAllRanges();

```

24.2 A API `Clipboard` (Moderna)

A API Clipboard moderna é assíncrona e mais segura que o antigo `document.execCommand`.

```
// Copiando texto para a área de transferência
async function copiarTexto(texto) {
  try {
    await navigator.clipboard.writeText(texto);
    console.log("Texto copiado com sucesso!");
  } catch (err) {
    console.error("Falha ao copiar:", err);
  }
}

// Lendo da área de transferência
async function lerAreaTransferencia() {
  try {
    const texto = await navigator.clipboard.readText();
    console.log(`Área de transferência: "${texto}"`);
    return texto;
  } catch (err) {
    console.error("Falha ao ler:", err);
  }
}

// Botão de copiar código
document.querySelectorAll(".btn-copiar-codigo").forEach(btn => {
  btn.addEventListener("click", async function() {
    const bloco = this.closest(".bloco-codigo");
    const codigo = bloco.querySelector("code").textContent;

    await copiarTexto(codigo);

    this.textContent = "Copiado!";
    setTimeout(() => { this.textContent = "Copiar"; }, 2000);
  });
});
```

24.3 A API **Fullscreen**

```
const video = document.querySelector("video");
const btnFullscreen = document.querySelector("#btn-fullscreen");

btnFullscreen.addEventListener("click", async function() {
  if (!document.fullscreenElement) {
    // Entrar em tela cheia
    try {
      await video.requestFullscreen();
      // ou: await document.documentElement.requestFullscreen(); // Página inteira
    } catch (err) {
      console.error(`Erro ao entrar em tela cheia: ${err.message}`);
    }
  } else {
    // Sair da tela cheia
    await document.exitFullscreen();
  }
});

// Detectando mudanças no estado de tela cheia
document.addEventListener("fullscreenchange", function() {
  if (document.fullscreenElement) {
    console.log("Entrou em tela cheia:", document.fullscreenElement);
    btnFullscreen.textContent = "Sair da Tela Cheia";
  } else {
    console.log("Saiu da tela cheia");
    btnFullscreen.textContent = "Tela Cheia";
  }
});
```


24.4 A API **Notification**

```

async function solicitarPermissaoNotificacao() {
  if (!("Notification" in window)) {
    console.log("Este navegador não suporta notificações.");
    return;
  }

  const permissao = await Notification.requestPermission();
  console.log(`Permissão: ${permissao}`); // "granted" | "denied" | "default"
  return permissao;
}

function enviarNotificacao(titulo, opcoes = {}) {
  if (Notification.permission === "granted") {
    const notificacao = new Notification(titulo, {
      body: opcoes.corpo || "",
      icon: opcoes.icone || "/icone.png",
      badge: opcoes.badge || "/badge.png",
      tag: opcoes.tag || "padrao",
      requireInteraction: opcoes.persistente || false
    });

    notificacao.onclick = function() {
      window.focus();
      notificacao.close();
    };

    // Fechar automaticamente após 5 segundos
    setTimeout(() => notificacao.close(), 5000);

    return notificacao;
  }
}

// Uso
solicitarPermissaoNotificacao().then(permissao => {
  if (permissao === "granted") {
    enviarNotificacao("Nova Mensagem!", {
      corpo: "Você recebeu uma nova mensagem de João.",

```

```
        icone: "/avatar-joao.png"  
      });  
    }  
  });
```

Capítulo 25: Padrões e Boas Práticas

25.1 O Padrão Module (IIFE)

Para evitar poluição do escopo global, encapsule seu código em módulos:

```
// IIFE (Immediately Invoked Function Expression)
const MinhaApp = (function() {
  // Variáveis privadas
  let _contador = 0;
  const _elementos = [];

  // Funções privadas
  function _validar(valor) {
    return valor !== null && valor !== undefined;
  }

  // Interface pública
  return {
    incrementar() {
      _contador++;
      return _contador;
    },

    obterContador() {
      return _contador;
    },

    adicionarElemento(el) {
      if (_validar(el)) {
        _elementos.push(el);
      }
    }
  };
})();

// Uso
MinhaApp.incrementar();
console.log(MinhaApp.obterContador()); // 1
// MinhaApp._contador // undefined (privado!)
```

25.2 Padrão Observer (Pub/Sub)

```

class EventEmitter {
  constructor() {
    this._eventos = {};
  }

  on(evento, callback) {
    if (!this._eventos[evento]) {
      this._eventos[evento] = [];
    }
    this._eventos[evento].push(callback);
    return this; // Para encadeamento
  }

  off(evento, callback) {
    if (this._eventos[evento]) {
      this._eventos[evento] = this._eventos[evento]
        .filter(cb => cb !== callback);
    }
    return this;
  }

  emit(evento, ...args) {
    if (this._eventos[evento]) {
      this._eventos[evento].forEach(cb => cb(...args));
    }
    return this;
  }

  once(evento, callback) {
    const wrapper = (...args) => {
      callback(...args);
      this.off(evento, wrapper);
    };
    return this.on(evento, wrapper);
  }
}

// Uso

```

```
const emitter = new EventEmitter();

emitter.on("dados:carregados", (dados) => {
  console.log("Dados recebidos:", dados);
  renderizarTabela(dados);
});

emitter.on("dados:carregados", (dados) => {
  atualizarGrafico(dados);
});

emitter.once("app:pronta", () => {
  console.log("App inicializada! Este handler só executa uma vez.");
});

// Em algum lugar do código:
emitter.emit("dados:carregados", { usuarios: [...] });
emitter.emit("app:pronta");
```

25.3 Otimização de Performance no DOM

Manipulações frequentes do DOM podem causar lentidão. Aqui estão as principais técnicas de otimização:

1. Minimize Reflows e Repaints

Um "reflow" ocorre quando o navegador precisa recalcular o layout (posição e tamanho dos elementos). Um "repaint" ocorre quando apenas a aparência visual muda (cor, fundo). Reflows são muito mais custosos.


```
// RUIM: Lê e escreve alternadamente, causando múltiplos reflows
const caixa = document.querySelector(".caixa");
caixa.style.width = caixa.offsetWidth + 10 + "px"; // Lê, depois escreve
caixa.style.height = caixa.offsetHeight + 10 + "px"; // Lê, depois escreve

// BOM: Lê tudo primeiro, depois escreve tudo
const largura = caixa.offsetWidth; // Lê
const altura = caixa.offsetHeight; // Lê
caixa.style.width = largura + 10 + "px"; // Escreve
caixa.style.height = altura + 10 + "px"; // Escreve
```

2. Use Classes CSS em vez de Estilos Inline para Mudanças Complexas

```
// RUIM: Múltiplas mudanças de estilo inline
elemento.style.color = "red";
elemento.style.fontSize = "20px";
elemento.style.fontWeight = "bold";
elemento.style.textDecoration = "underline";

// BOM: Uma única mudança de classe
elemento.classList.add("destaque-erro");
// No CSS: .destaque-erro { color: red; font-size: 20px; font-weight: bold; text-decoration: underline; }
```

3. Use `DocumentFragment` para Inserções em Lote

```
// RUIM: Insere um elemento por vez no DOM real
const lista = document.querySelector("ul");
dados.forEach(item => {
  const li = document.createElement("li");
  li.textContent = item.nome;
  lista.appendChild(li); // Reflow a cada inserção!
});

// BOM: Usa DocumentFragment
const fragment = document.createDocumentFragment();
dados.forEach(item => {
  const li = document.createElement("li");
  li.textContent = item.nome;
  fragment.appendChild(li); // Sem reflow
});
lista.appendChild(fragment); // Um único reflow
```

25.4 Acessibilidade (a11y) e o DOM

Criar interfaces acessíveis é uma responsabilidade importante do desenvolvedor Frontend.

```
// Gerenciando foco para acessibilidade
const modal = document.querySelector("#modal");
const btnAbrirModal = document.querySelector("#btn-abrir-modal");
const btnFecharModal = modal.querySelector(".btn-fechar");

function abrirModal() {
    modal.style.display = "flex";
    modal.setAttribute("aria-hidden", "false");

    // Move o foco para dentro do modal
    btnFecharModal.focus();

    // Captura o foco dentro do modal (trap focus)
    modal.addEventListener("keydown", capturarFoco);
}

function fecharModal() {
    modal.style.display = "none";
    modal.setAttribute("aria-hidden", "true");

    // Devolve o foco ao botão que abriu o modal
    btnAbrirModal.focus();

    modal.removeEventListener("keydown", capturarFoco);
}

function capturarFoco(e) {
    if (e.key !== "Tab") return;

    const focaveis = modal.querySelectorAll(
        'button, [href], input, select, textarea, [tabindex]:not([tabindex="-1"])'
    );
    const primeiro = focaveis[0];
    const ultimo = focaveis[focaveis.length - 1];

    if (e.shiftKey) {
        if (document.activeElement === primeiro) {
            e.preventDefault();
        }
    }
}
```

```
        ultimo.focus();
    }
} else {
    if (document.activeElement === ultimo) {
        e.preventDefault();
        primeiro.focus();
    }
}
}

// Anunciando mudanças dinâmicas para leitores de tela (ARIA Live Regions)
const regiaoAoVivo = document.querySelector("[aria-live='polite']");

function anunciarMensagem(mensagem) {
    regiaoAoVivo.textContent = mensagem;
    // Limpar após um tempo para permitir reanúncio da mesma mensagem
    setTimeout(() => { regiaoAoVivo.textContent = ""; }, 1000);
}
```

Capítulo 26: Exercícios Resolvidos

Exercício 1: Galeria de Imagens com Lightbox

Objetivo: Criar uma galeria de imagens onde clicar em uma miniatura abre uma versão ampliada em um overlay.

```

// HTML esperado:
// <div class="galeria">
//   
//   
// </div>
// <div id="lightbox" class="lightbox">
//   <button class="fechar">&times;</button>
//   <button class="anterior">&#10094;</button>
//   <img id="lightbox-img" src="" alt="">
//   <button class="proximo">&#10095;</button>
// </div>

const galeria = document.querySelector(".galeria");
const lightbox = document.getElementById("lightbox");
const lightboxImg = document.getElementById("lightbox-img");
const imagens = Array.from(galeria.querySelectorAll("img"));
let indiceAtual = 0;

function abrirLightbox(indice) {
  indiceAtual = indice;
  lightboxImg.src = imagens[indice].dataset.full;
  lightboxImg.alt = imagens[indice].alt;
  lightbox.classList.add("ativo");
  document.body.style.overflow = "hidden";
  lightbox.querySelector(".fechar").focus();
}

function fecharLightbox() {
  lightbox.classList.remove("ativo");
  document.body.style.overflow = "";
  imagens[indiceAtual].focus();
}

function navegarLightbox(direcao) {
  indiceAtual = (indiceAtual + direcao + imagens.length) % imagens.length;
  lightboxImg.src = imagens[indiceAtual].dataset.full;
  lightboxImg.alt = imagens[indiceAtual].alt;
}

```

```
// Delegação de eventos na galeria
galeria.addEventListener("click", function(e) {
  if (e.target.tagName === "IMG") {
    abrirLightbox(imagens.indexOf(e.target));
  }
});

lightbox.querySelector(".fechar").addEventListener("click", fecharLightbox);
lightbox.querySelector(".anterior").addEventListener("click", () => navegarLightbox(-1));
lightbox.querySelector(".proximo").addEventListener("click", () => navegarLightbox(1))

// Fechar ao clicar fora da imagem
lightbox.addEventListener("click", function(e) {
  if (e.target === lightbox) fecharLightbox();
});

// Navegação por teclado
document.addEventListener("keydown", function(e) {
  if (!lightbox.classList.contains("ativo")) return;
  if (e.key === "Escape") fecharLightbox();
  if (e.key === "ArrowLeft") navegarLightbox(-1);
  if (e.key === "ArrowRight") navegarLightbox(1);
});
```

Exercício 2: Autocompletar (Autocomplete)


```

const dados = ["JavaScript", "Java", "Python", "PHP", "Ruby", "Go", "Rust", "TypeScript", "C++", "C"];

const input = document.querySelector("#busca-linguagem");
const listaSugestoes = document.querySelector("#sugestoes");
let indiceSelecioneado = -1;

input.addEventListener("input", function() {
    const termo = this.value.toLowerCase().trim();
    listaSugestoes.innerHTML = "";
    indiceSelecioneado = -1;

    if (!termo) {
        listaSugestoes.style.display = "none";
        return;
    }

    const filtrados = dados.filter(item => item.toLowerCase().startsWith(termo));

    if (filtrados.length === 0) {
        listaSugestoes.style.display = "none";
        return;
    }

    filtrados.forEach((item, i) => {
        const li = document.createElement("li");
        // Destaca a parte que corresponde ao termo buscado
        li.innerHTML = `<strong>${item.substring(0, termo.length)}</strong>${item.substring(termo.length)}`;
        li.dataset.valor = item;
        li.addEventListener("click", () => selecionarItem(item));
        listaSugestoes.appendChild(li);
    });

    listaSugestoes.style.display = "block";
});

input.addEventListener("keydown", function(e) {
    const itens = listaSugestoes.querySelectorAll("li");

```

```
if (e.key === "ArrowDown") {
    e.preventDefault();
    indiceSelecioneado = Math.min(indiceSelecioneado + 1, itens.length - 1);
    atualizarSelecao(itens);
} else if (e.key === "ArrowUp") {
    e.preventDefault();
    indiceSelecioneado = Math.max(indiceSelecioneado - 1, -1);
    atualizarSelecao(itens);
} else if (e.key === "Enter" && indiceSelecioneado >= 0) {
    e.preventDefault();
    selecionarItem(itens[indiceSelecioneado].dataset.valor);
} else if (e.key === "Escape") {
    listaSugestoes.style.display = "none";
}
});

function atualizarSelecao(itens) {
    itens.forEach((item, i) => {
        item.classList.toggle("selecioneado", i === indiceSelecioneado);
    });
}

function selecionarItem(valor) {
    input.value = valor;
    listaSugestoes.style.display = "none";
    input.focus();
}

// Fechar ao clicar fora
document.addEventListener("click", function(e) {
    if (!e.target.closest(".container-autocomplete")) {
        listaSugestoes.style.display = "none";
    }
});
```

Capítulo 27: Referência de Propriedades do Objeto Event

27.1 Propriedades Comuns a Todos os Eventos

Propriedade	Tipo	Descrição
<code>type</code>	String	Nome do evento (ex: "click", "keydown")
<code>target</code>	Element	Elemento que disparou o evento
<code>currentTarget</code>	Element	Elemento ao qual o listener está anexado
<code>bubbles</code>	Boolean	Se o evento faz bubble
<code>cancelable</code>	Boolean	Se pode ser cancelado com <code>preventDefault()</code>
<code>defaultPrevented</code>	Boolean	Se <code>preventDefault()</code> foi chamado
<code>eventPhase</code>	Number	Fase atual: 1=Captura, 2=Alvo, 3=Bubbling
<code>timestamp</code>	Number	Tempo em ms desde a origem
<code>isTrusted</code>	Boolean	Se foi gerado pelo usuário (true) ou por script (false)

27.2 Propriedades Específicas de Eventos de Mouse

Propriedade	Tipo	Descrição
<code>clientX</code> / <code>clientY</code>	Number	Coordenadas relativas à viewport
<code>pageX</code> / <code>pageY</code>	Number	Coordenadas relativas ao documento
<code>screenX</code> / <code>screenY</code>	Number	Coordenadas relativas à tela
<code>offsetX</code> / <code>offsetY</code>	Number	Coordenadas relativas ao elemento alvo
<code>movementX</code> / <code>movementY</code>	Number	Movimento desde o último evento
<code>button</code>	Number	Botão pressionado: 0=esq, 1=meio, 2=dir
<code>buttons</code>	Number	Bitmask dos botões pressionados
<code>ctrlKey</code>	Boolean	Ctrl estava pressionado
<code>shiftKey</code>	Boolean	Shift estava pressionado
<code>altKey</code>	Boolean	Alt estava pressionado
<code>metaKey</code>	Boolean	Meta (Cmd/Win) estava pressionado
<code>relatedTarget</code>	Element	Elemento relacionado (para enter/leave)
<code>detail</code>	Number	Contagem de cliques

27.3 Propriedades Específicas de Eventos de Teclado

Propriedade	Tipo	Descrição
<code>key</code>	String	Valor da tecla (ex: "a", "Enter", "ArrowUp")
<code>code</code>	String	Código físico da tecla (ex: "KeyA", "Enter")
<code>keyCode</code>	Number	Código numérico (OBSOLETO)
<code>charCode</code>	Number	Código do caractere (OBSOLETO)
<code>which</code>	Number	Código da tecla (OBSOLETO)
<code>ctrlKey</code>	Boolean	Ctrl estava pressionado
<code>shiftKey</code>	Boolean	Shift estava pressionado
<code>altKey</code>	Boolean	Alt estava pressionado
<code>metaKey</code>	Boolean	Meta estava pressionado
<code>repeat</code>	Boolean	Se a tecla está sendo mantida pressionada
<code>isComposing</code>	Boolean	Se está em composição de caracteres (IME)

27.4 Propriedades Específicas de Eventos de Toque

Propriedade	Tipo	Descrição
<code>touches</code>	TouchList	Todos os pontos de toque ativos
<code>targetTouches</code>	TouchList	Toques no elemento alvo
<code>changedTouches</code>	TouchList	Toques que mudaram neste evento
<code>touch.identifier</code>	Number	ID único do ponto de toque
<code>touch.clientX/Y</code>	Number	Coordenadas do toque na viewport
<code>touch.pageX/Y</code>	Number	Coordenadas do toque no documento
<code>touch.radiusX/Y</code>	Number	Raio da área de contato
<code>touch.force</code>	Number	Pressão do toque (0 a 1)

27.5 Propriedades do Evento de Roda do Mouse (`wheel`)

Propriedade	Tipo	Descrição
<code>deltaX</code>	Number	Scroll horizontal
<code>deltaY</code>	Number	Scroll vertical
<code>deltaZ</code>	Number	Scroll de profundidade
<code>deltaMode</code>	Number	Unidade: 0=pixels, 1=linhas, 2=páginas

Capítulo 28: Guia de Depuração do DOM

28.1 Ferramentas do Desenvolvedor do Navegador

O painel "Elements" (ou "Inspector") do DevTools é sua principal ferramenta para inspecionar e depurar o DOM. Você pode:

- Inspecionar a estrutura da árvore do DOM em tempo real.
- Editar HTML e atributos diretamente no painel.
- Ver os estilos computados de qualquer elemento.
- Adicionar breakpoints no DOM para pausar a execução quando o DOM mudar.

28.2 Técnicas de Depuração via Console

```
// Inspecionar um elemento no console
const elemento = document.querySelector(".meu-elemento");
console.log(elemento);           // Exibe o elemento HTML
console.dir(elemento);          // Exibe o objeto JavaScript com todas as propriedades

// Verificar o tipo de um nó
console.log(elemento.nodeType, elemento.nodeName);

// Listar todas as propriedades de um elemento
console.log(Object.getOwnPropertyNames(elemento));

// Monitorar eventos em um elemento (apenas no DevTools)
// monitorEvents(elemento, "click"); // Mostra todos os eventos de clique
// unmonitorEvents(elemento);

// Inspecionar os listeners de um elemento (apenas no DevTools)
// getEventListeners(elemento); // Retorna objeto com todos os listeners

// Medir performance de operações DOM
console.time("operacao-dom");
for (let i = 0; i < 1000; i++) {
    document.querySelector(".item");
}
console.timeEnd("operacao-dom"); // Exibe o tempo decorrido

// Agrupando logs relacionados
console.group("Inicialização do DOM");
console.log("Selecionando elementos...");
console.log("Adicionando eventos...");
console.groupEnd();
```


28.3 Breakpoints no DOM

O DevTools permite adicionar "DOM breakpoints" que pausam a execução do JavaScript quando:

- Um nó filho é adicionado ou removido (subtree modifications).
- Um atributo do elemento é alterado (attribute modifications).
- O próprio nó é removido (node removal).

Para adicionar um DOM breakpoint: clique com o botão direito em um elemento no painel Elements > Break on > escolha o tipo.

28.4 Erros Comuns e Como Corrigi-los

Erro	Causa Provável	Solução
<code>Cannot read properties of null</code>	Elemento não encontrado no DOM	Verificar se o seletor está correto e se o DOM está pronto
<code>element.addEventListener is not a function</code>	A variável não é um elemento DOM	Verificar o retorno do seletor com <code>console.log()</code>
Evento não dispara	Listener adicionado ao elemento errado	Usar <code>console.log(e.target)</code> para verificar
Evento dispara múltiplas vezes	Listener adicionado múltiplas vezes	Remover o listener antes de adicionar novamente
<code>innerHTML</code> não atualiza	Referência ao elemento perdida	Reobter a referência após modificar o DOM pai
Estilos não aplicados	Especificidade CSS	Usar <code>getComputedStyle()</code> para verificar o estilo final
<code>localStorage</code> vazio	Serialização incorreta	Verificar uso de <code>JSON.stringify/parse</code>

Capítulo 29: Projeto Completo — Painel de Administração

29.1 Visão Geral do Projeto

Neste capítulo, construímos um painel de administração completo que integra todos os conceitos aprendidos ao longo da apostila. O projeto inclui: tabela de dados com CRUD, filtros, ordenação, paginação, modal de edição, notificações toast e persistência via `localStorage`.

29.2 Estrutura HTML do Painel

```
<!DOCTYPE html>
<html lang="pt-BR">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Painel de Usuários</title>
  <style>
    * { box-sizing: border-box; margin: 0; padding: 0; }
    body { font-family: Arial, sans-serif; background: #f5f6fa; color: #333; }

    .header {
      background: #2c3e50;
      color: white;
      padding: 16px 24px;
      display: flex;
      justify-content: space-between;
      align-items: center;
    }

    .container { max-width: 1200px; margin: 24px auto; padding: 0 24px; }

    .toolbar {
      display: flex;
      gap: 12px;
      margin-bottom: 16px;
      flex-wrap: wrap;
    }

    .toolbar input {
      flex: 1;
      min-width: 200px;
      padding: 8px 12px;
      border: 1px solid #ddd;
      border-radius: 4px;
      font-size: 14px;
    }

    .btn {
```

```

        padding: 8px 16px;
        border: none;
        border-radius: 4px;
        cursor: pointer;
        font-size: 14px;
        font-weight: 600;
    }

    .btn-primario { background: #3498db; color: white; }
    .btn-sucesso { background: #2ecc71; color: white; }
    .btn-perigo { background: #e74c3c; color: white; }
    .btn-aviso { background: #f39c12; color: white; }

    table { width: 100%; border-collapse: collapse; background: white; border-radius: 8px; }
    thead { background: #2c3e50; color: white; }
    th { padding: 12px 16px; text-align: left; cursor: pointer; user-select: none; }
    th:hover { background: #34495e; }
    td { padding: 12px 16px; border-bottom: 1px solid #eee; }
    tr:last-child td { border-bottom: none; }
    tr:hover td { background: #f8f9fa; }

    .badge {
        padding: 3px 8px;
        border-radius: 12px;
        font-size: 12px;
        font-weight: 600;
    }

    .badge-ativo { background: #d5f5e3; color: #1e8449; }
    .badge-inativo { background: #fadbd8; color: #922b21; }

    .paginacao {
        display: flex;
        justify-content: center;
        gap: 8px;
        margin-top: 16px;
    }

    .paginacao button {
        width: 36px;

```

```

        height: 36px;
        border: 1px solid #ddd;
        border-radius: 4px;
        cursor: pointer;
        background: white;
    }

    .paginacao button.ativo { background: #3498db; color: white; border-color: #3498db; }

    /* Modal */
    .overlay {
        display: none;
        position: fixed;
        inset: 0;
        background: rgba(0,0,0,0.5);
        z-index: 1000;
        align-items: center;
        justify-content: center;
    }
    .overlay.ativo { display: flex; }
    .modal {
        background: white;
        border-radius: 8px;
        padding: 24px;
        width: 100%;
        max-width: 480px;
        box-shadow: 0 10px 30px rgba(0,0,0,0.3);
    }
    .modal h2 { margin-bottom: 20px; color: #2c3e50; }
    .campo { margin-bottom: 16px; }
    .campo label { display: block; margin-bottom: 6px; font-weight: 600; font-size: 14px; }
    .campo input, .campo select {
        width: 100%;
        padding: 8px 12px;
        border: 1px solid #ddd;
        border-radius: 4px;
        font-size: 14px;
    }
    .modal-acoas { display: flex; gap: 12px; justify-content: flex-end; margin-top: 20px; }

```

```

/* Toast */
.toast-container {
  position: fixed;
  bottom: 24px;
  right: 24px;
  display: flex;
  flex-direction: column;
  gap: 8px;
  z-index: 2000;
}
.toast {
  padding: 12px 20px;
  border-radius: 6px;
  color: white;
  font-size: 14px;
  box-shadow: 0 4px 12px rgba(0,0,0,0.2);
  animation: slideIn 0.3s ease;
  max-width: 320px;
}
.toast-sucesso { background: #2ecc71; }
.toast-erro { background: #e74c3c; }
.toast-info { background: #3498db; }
@keyframes slideIn { from { transform: translateX(100%); opacity: 0; } to { transform: tra
</style>
</head>
<body>
  <header class="header">
    <h1>Painel de Usuários</h1>
    <button class="btn btn-sucesso" id="btn-novo">+ Novo Usuário</button>
  </header>

  <main class="container">
    <div class="toolbar">
      <input type="text" id="busca" placeholder="Buscar por nome ou e-mail...">
      <select id="filtro-status">
        <option value="">Todos os status</option>
        <option value="ativo">Ativo</option>
        <option value="inativo">Inativo</option>

```

```

    </select>
    <select id="filtro-departamento">
      <option value="">Todos os departamentos</option>
      <option value="TI">TI</option>
      <option value="RH">RH</option>
      <option value="Financeiro">Financeiro</option>
      <option value="Marketing">Marketing</option>
    </select>
  </div>

  <table id="tabela-usuarios">
    <thead>
      <tr>
        <th data-coluna="id">ID ‡</th>
        <th data-coluna="nome">Nome ‡</th>
        <th data-coluna="email">E-mail ‡</th>
        <th data-coluna="departamento">Departamento ‡</th>
        <th data-coluna="status">Status ‡</th>
        <th>Ações</th>
      </tr>
    </thead>
    <tbody id="corpo-tabela"></tbody>
  </table>

  <div class="paginacao" id="paginacao"></div>
  <p id="info-registros" style="text-align:center; margin-top:12px; color:#666; font-size:14px">
</main>

<div class="overlay" id="overlay-modal">
  <div class="modal" role="dialog" aria-labelledby="titulo-modal">
    <h2 id="titulo-modal">Novo Usuário</h2>
    <form id="form-usuario">
      <div class="campo">
        <label for="campo-nome">Nome Completo</label>
        <input type="text" id="campo-nome" name="nome" required>
      </div>
      <div class="campo">
        <label for="campo-email">E-mail</label>
        <input type="email" id="campo-email" name="email" required>

```



```

    </div>
    <div class="campo">
      <label for="campo-departamento">Departamento</label>
      <select id="campo-departamento" name="departamento" required>
        <option value="">Selecione...</option>
        <option value="TI">TI</option>
        <option value="RH">RH</option>
        <option value="Financeiro">Financeiro</option>
        <option value="Marketing">Marketing</option>
      </select>
    </div>
    <div class="campo">
      <label for="campo-status">Status</label>
      <select id="campo-status" name="status">
        <option value="ativo">Ativo</option>
        <option value="inativo">Inativo</option>
      </select>
    </div>
    <div class="modal-acoes">
      <button type="button" class="btn" id="btn-cancelar">Cancelar</button>
      <button type="submit" class="btn btn-primario">Salvar</button>
    </div>
  </form>
</div>
</div>

<div class="toast-container" id="toasts"></div>

<script src="painel.js"></script>
</body>
</html>

```

29.3 JavaScript do Painel (`painel.js`)

```

// =====
// DADOS E ESTADO
// =====

const USUARIOS_INICIAIS = [
  { id: 1, nome: "Alice Ferreira", email: "alice@empresa.com", departamento: "TI", status: "ativo" },
  { id: 2, nome: "Bruno Santos", email: "bruno@empresa.com", departamento: "RH", status: "ativo" },
  { id: 3, nome: "Carla Mendes", email: "carla@empresa.com", departamento: "TI", status: "inativo" },
  { id: 4, nome: "Diego Lima", email: "diego@empresa.com", departamento: "Financeiro", status: "ativo" },
  { id: 5, nome: "Elena Costa", email: "elena@empresa.com", departamento: "Marketing", status: "ativo" },
  { id: 6, nome: "Fábio Rocha", email: "fabio@empresa.com", departamento: "RH", status: "inativo" },
  { id: 7, nome: "Gabi Alves", email: "gabi@empresa.com", departamento: "TI", status: "ativo" },
  { id: 8, nome: "Hugo Nunes", email: "hugo@empresa.com", departamento: "Financeiro", status: "ativo" },
  { id: 9, nome: "Íris Pinto", email: "iris@empresa.com", departamento: "Marketing", status: "ativo" },
  { id: 10, nome: "João Vieira", email: "joao@empresa.com", departamento: "TI", status: "inativo" },
  { id: 11, nome: "Karen Dias", email: "karen@empresa.com", departamento: "RH", status: "ativo" },
  { id: 12, nome: "Lucas Melo", email: "lucas@empresa.com", departamento: "TI", status: "ativo" }
];

let usuarios = JSON.parse(localStorage.getItem("usuarios")) || USUARIOS_INICIAIS;
let proximoId = Math.max(...usuarios.map(u => u.id)) + 1;
let usuarioEditando = null;

const estado = {
  busca: "",
  filtroStatus: "",
  filtroDepartamento: "",
  colunaOrdenacao: "id",
  direcaoOrdenacao: "asc",
  paginaAtual: 1,
  itensPorPagina: 5
};

// =====
// PERSISTÊNCIA
// =====

function salvarDados() {
  localStorage.setItem("usuarios", JSON.stringify(usuarios));
}

```

```

// =====
// FILTROS E ORDENAÇÃO
// =====
function obterUsuariosFiltrados() {
    return usuarios
        .filter(u => {
            const termoBusca = estado.busca.toLowerCase();
            const correspondeBusca = !termoBusca ||
                u.nome.toLowerCase().includes(termoBusca) ||
                u.email.toLowerCase().includes(termoBusca);
            const correspondeStatus = !estado.filtroStatus || u.status === estado.filtroStatus;
            const correspondeDept = !estado.filtroDepartamento || u.departamento === estado.filtroDepartamento;
            return correspondeBusca && correspondeStatus && correspondeDept;
        })
        .sort((a, b) => {
            const valA = a[estado.colunaOrdenacao];
            const valB = b[estado.colunaOrdenacao];
            const comp = typeof valA === "string" ? valA.localeCompare(valB) : valA - valB;
            return estado.direcaoOrdenacao === "asc" ? comp : -comp;
        });
}

// =====
// RENDERIZAÇÃO
// =====
function renderizar() {
    const filtrados = obterUsuariosFiltrados();
    const totalPaginas = Math.ceil(filtrados.length / estado.itensPorPagina);
    estado.paginaAtual = Math.min(estado.paginaAtual, totalPaginas || 1);

    const inicio = (estado.paginaAtual - 1) * estado.itensPorPagina;
    const paginados = filtrados.slice(inicio, inicio + estado.itensPorPagina);

    const tbody = document.getElementById("corpo-tabela");
    tbody.innerHTML = "";

    if (paginados.length === 0) {
        tbody.innerHTML = `<tr><td colspan="6" style="text-align:center; padding:40px; color:#999;`
    }
}

```

```

    } else {
      paginados.forEach(u => {
        const tr = document.createElement("tr");
        tr.innerHTML = `
          <td>${u.id}</td>
          <td><strong>${u.nome}</strong></td>
          <td>${u.email}</td>
          <td>${u.departamento}</td>
          <td><span class="badge badge-${u.status}">${u.status === "ativo" ? "Ativo" : "Inativo"}</span></td>
          <td>
            <button class="btn btn-aviso btn-editar" data-id="${u.id}" style="padding:4px 10px">Aviso</button>
            <button class="btn btn-perigo btn-excluir" data-id="${u.id}" style="padding:4px 10px">Excluir</button>
          </td>
        `;
        tbody.appendChild(tr);
      });
    }
  }

  renderizarPaginacao(totalPaginas);

  const fim = Math.min(inicio + estado.itensPorPagina, filtrados.length);
  document.getElementById("info-registros").textContent =
    filtrados.length > 0
      ? `Exibindo ${inicio + 1}-${fim} de ${filtrados.length} registros`
      : "Nenhum registro encontrado";
}

function renderizarPaginacao(total) {
  const container = document.getElementById("paginacao");
  container.innerHTML = "";
  if (total <= 1) return;

  const criarBtn = (texto, pagina, ativo = false, desabilitado = false) => {
    const btn = document.createElement("button");
    btn.textContent = texto;
    if (ativo) btn.classList.add("ativo");
    btn.disabled = desabilitado;
    if (!desabilitado) btn.addEventListener("click", () => { estado.paginaAtual = pagina; renderizar(); });
    return btn;
  };

```

```

};

container.appendChild(criarBtn("<", 1, false, estado.paginaAtual === 1));
container.appendChild(criarBtn("<", estado.paginaAtual - 1, false, estado.paginaAtual === 1));

for (let i = 1; i <= total; i++) {
    container.appendChild(criarBtn(i, i, i === estado.paginaAtual));
}

container.appendChild(criarBtn(">", estado.paginaAtual + 1, false, estado.paginaAtual === total));
container.appendChild(criarBtn(">", total, false, estado.paginaAtual === total));
}

// =====
// MODAL
// =====
function abrirModal(usuario = null) {
    usuarioEditando = usuario;
    const titulo = document.getElementById("titulo-modal");
    const form = document.getElementById("form-usuario");

    titulo.textContent = usuario ? "Editar Usuário" : "Novo Usuário";
    form.reset();

    if (usuario) {
        form.nome.value = usuario.nome;
        form.email.value = usuario.email;
        form.departamento.value = usuario.departamento;
        form.status.value = usuario.status;
    }

    document.getElementById("overlay-modal").classList.add("ativo");
    document.getElementById("campo-nome").focus();
}

function fecharModal() {
    document.getElementById("overlay-modal").classList.remove("ativo");
    usuarioEditando = null;
}

```

```

// =====
// CRUD
// =====

function salvarUsuario(dados) {
  if (usuarioEditando) {
    Object.assign(usuarioEditando, dados);
    mostrarToast("Usuário atualizado com sucesso!", "sucesso");
  } else {
    usuarios.push({ id: proximoId++, ...dados });
    mostrarToast("Usuário criado com sucesso!", "sucesso");
  }
  salvarDados();
  fecharModal();
  renderizar();
}

function excluirUsuario(id) {
  if (!confirm("Tem certeza que deseja excluir este usuário?")) return;
  usuarios = usuarios.filter(u => u.id !== id);
  salvarDados();
  renderizar();
  mostrarToast("Usuário excluído.", "info");
}

// =====
// TOAST
// =====

function mostrarToast(mensagem, tipo = "info") {
  const container = document.getElementById("toasts");
  const toast = document.createElement("div");
  toast.className = `toast toast-${tipo}`;
  toast.textContent = mensagem;
  container.appendChild(toast);
  setTimeout(() => toast.remove(), 3500);
}

// =====
// EVENT LISTENERS

```

```
// =====
document.getElementById("btn-novo").addEventListener("click", () => abrirModal());
document.getElementById("btn-cancelar").addEventListener("click", fecharModal);

document.getElementById("overlay-modal").addEventListener("click", function(e) {
  if (e.target === this) fecharModal();
});

document.addEventListener("keydown", function(e) {
  if (e.key === "Escape") fecharModal();
});

document.getElementById("form-usuario").addEventListener("submit", function(e) {
  e.preventDefault();
  salvarUsuario({
    nome: this.nome.value.trim(),
    email: this.email.value.trim(),
    departamento: this.departamento.value,
    status: this.status.value
  });
});

// Delegação de eventos na tabela
document.getElementById("corpo-tabela").addEventListener("click", function(e) {
  const id = parseInt(e.target.dataset.id);
  if (e.target.classList.contains("btn-editar")) {
    abrirModal(usuarios.find(u => u.id === id));
  }
  if (e.target.classList.contains("btn-excluir")) {
    excluirUsuario(id);
  }
});

// Ordenação por coluna
document.querySelectorAll("th[data-coluna]").forEach(th => {
  th.addEventListener("click", function() {
    const coluna = this.dataset.coluna;
    if (estado.colunaOrdenacao === coluna) {
      estado.direcaoOrdenacao = estado.direcaoOrdenacao === "asc" ? "desc" : "asc";
    }
  });
});
```



```
    } else {
        estado.colunaOrdenacao = coluna;
        estado.direcaoOrdenacao = "asc";
    }
    estado.paginaAtual = 1;
    renderizar();
});

});

// Filtros com debounce
function debounce(fn, delay) {
    let timer;
    return (...args) => { clearTimeout(timer); timer = setTimeout(() => fn(...args), delay); };
}

document.getElementById("busca").addEventListener("input", debounce(function() {
    estado.busca = this.value;
    estado.paginaAtual = 1;
    renderizar();
}, 300)));

document.getElementById("filtro-status").addEventListener("change", function() {
    estado.filtroStatus = this.value;
    estado.paginaAtual = 1;
    renderizar();
});

document.getElementById("filtro-departamento").addEventListener("change", function() {
    estado.filtroDepartamento = this.value;
    estado.paginaAtual = 1;
    renderizar();
});

// Inicialização
renderizar();
```

Capítulo 30: Referência Completa de Propriedades do Elemento DOM

30.1 Propriedades de Identificação e Estrutura

Propriedade	Tipo	Descrição
<code>id</code>	String	O atributo <code>id</code> do elemento
<code>tagName</code>	String	Nome da tag em maiúsculas (ex: "DIV")
<code>nodeName</code>	String	Nome do nó (igual a <code>tagName</code> para elementos)
<code>nodeType</code>	Number	Tipo do nó (1=Element, 3=Text, 9=Document)
<code>nodeValue</code>	String/null	Valor do nó (null para elementos)
<code>className</code>	String	String com todas as classes
<code>classList</code>	DOMTokenList	Lista de classes com métodos
<code>attributes</code>	NamedNodeMap	Coleção de todos os atributos
<code>dataset</code>	DOMStringMap	Atributos <code>data-*</code>
<code>innerHTML</code>	String	Conteúdo HTML interno
<code>outerHTML</code>	String	HTML do elemento incluindo ele mesmo
<code>innerText</code>	String	Texto visível (consciente do CSS)
<code>textContent</code>	String	Texto completo (ignora CSS)

30.2 Propriedades de Navegação

Propriedade	Tipo	Descrição
<code>parentNode</code>	Node	Nó pai
<code>parentElement</code>	Element	Elemento pai
<code>children</code>	HTMLCollection	Filhos do tipo Element
<code>childNodes</code>	NodeList	Todos os nós filhos
<code>firstChild</code>	Node	Primeiro nó filho
<code>lastChild</code>	Node	Último nó filho
<code>firstElementChild</code>	Element	Primeiro filho Element
<code>lastElementChild</code>	Element	Último filho Element
<code>nextSibling</code>	Node	Próximo irmão (qualquer tipo)
<code>previousSibling</code>	Node	Irmão anterior (qualquer tipo)
<code>nextElementSibling</code>	Element	Próximo irmão Element
<code>previousElementSibling</code>	Element	Irmão anterior Element
<code>childElementCount</code>	Number	Número de filhos Element

30.3 Propriedades de Dimensão e Posição

Propriedade	Tipo	Inclui	Descrição
<code>clientWidth</code>	Number	Conteúdo + Padding	Largura interna visível
<code>clientHeight</code>	Number	Conteúdo + Padding	Altura interna visível
<code>clientTop</code>	Number	—	Largura da borda superior
<code>clientLeft</code>	Number	—	Largura da borda esquerda
<code>offsetWidth</code>	Number	Conteúdo + Padding + Border	Largura total
<code>offsetHeight</code>	Number	Conteúdo + Padding + Border	Altura total
<code>offsetTop</code>	Number	—	Distância ao <code>offsetParent</code> (vertical)
<code>offsetLeft</code>	Number	—	Distância ao <code>offsetParent</code> (horizontal)
<code>offsetParent</code>	Element	—	Ancestral posicionado mais próximo
<code>scrollWidth</code>	Number	Conteúdo total	Largura total do conteúdo
<code>scrollHeight</code>	Number	Conteúdo total	Altura total do conteúdo
<code>scrollTop</code>	Number	—	Pixels rolados verticalmente
<code>scrollLeft</code>	Number	—	Pixels rolados horizontalmente

30.4 Propriedades de Estado e Interação

Propriedade	Tipo	Aplicável a	Descrição
<code>hidden</code>	Boolean	Todos	Se o elemento está oculto (<code>display:none</code>)
<code>disabled</code>	Boolean	Form elements	Se o elemento está desabilitado
<code>checked</code>	Boolean	checkbox, radio	Se está marcado
<code>selected</code>	Boolean	option	Se está selecionado
<code>required</code>	Boolean	Form elements	Se é obrigatório
<code>readOnly</code>	Boolean	input, textarea	Se é somente leitura
<code>value</code>	String	Form elements	Valor atual
<code>defaultValue</code>	String	input, textarea	Valor padrão (do HTML)
<code>placeholder</code>	String	input, textarea	Texto de placeholder
<code>type</code>	String	input	Tipo do campo
<code>name</code>	String	Form elements	Atributo name
<code>href</code>	String	a, link	URL do link (absoluta)
<code>src</code>	String	img, script, etc.	URL do recurso (absoluta)
<code>alt</code>	String	img	Texto alternativo
<code>title</code>	String	Todos	Texto de tooltip
<code>tabIndex</code>	Number	Todos	Ordem de tabulação
<code>contentEditable</code>	String	Todos	"true", "false", "inherit"

Propriedade	Tipo	Aplicável a	Descrição
<code>isContentEditable</code>	Boolean	Todos	Se é editável (computado)
<code>draggable</code>	Boolean	Todos	Se pode ser arrastado
<code>lang</code>	String	Todos	Idioma do elemento
<code>dir</code>	String	Todos	Direção do texto ("ltr", "rtl")

30.5 Métodos Essenciais do Elemento

Método	Retorna	Descrição
<code>getAttribute(nome)</code>	String/null	Obtém valor do atributo
<code>setAttribute(nome, valor)</code>	void	Define valor do atributo
<code>removeAttribute(nome)</code>	void	Remove o atributo
<code>hasAttribute(nome)</code>	Boolean	Verifica se o atributo existe
<code>toggleAttribute(nome)</code>	Boolean	Alterna um atributo booleano
<code>querySelector(seletor)</code>	Element/null	Primeiro descendente que corresponde
<code>querySelectorAll(seletor)</code>	NodeList	Todos os descendentes que correspondem
<code>closest(seletor)</code>	Element/null	Ancestral mais próximo que corresponde
<code>matches(seletor)</code>	Boolean	Se o elemento corresponde ao seletor
<code>contains(no)</code>	Boolean	Se o nó é descendente
<code>appendChild(no)</code>	Node	Insere como último filho
<code>prepend(...nos)</code>	void	Insere como primeiro filho
<code>append(...nos)</code>	void	Insere como último filho
<code>insertBefore(novo, ref)</code>	Node	Insere antes do nó de referência
<code>replaceChild(novo, antigo)</code>	Node	Substitui um filho
<code>removeChild(no)</code>	Node	Remove um filho
<code>remove()</code>	void	Remove o próprio elemento
<code>replaceWith(...nos)</code>	void	Substitui o próprio elemento
<code>cloneNode(profundo)</code>	Node	Cria uma cópia

Método	Retorna	Descrição
<code>insertAdjacentHTML(pos, html)</code>	void	Insere HTML em posição específica
<code>insertAdjacentElement(pos, el)</code>	Element	Insere elemento em posição específica
<code>insertAdjacentText(pos, txt)</code>	void	Insere texto em posição específica
<code>getBoundingClientRect()</code>	DOMRect	Dimensões e posição na viewport
<code>getClientRects()</code>	DOMRectList	Retângulos de todas as caixas
<code>scrollIntoView(opcoes)</code>	void	Rola até o elemento ficar visível
<code>focus(opcoes)</code>	void	Foca o elemento
<code>blur()</code>	void	Remove o foco
<code>click()</code>	void	Simula um clique
<code>requestFullscreen()</code>	Promise	Entra em tela cheia
<code>animate(keyframes, opcoes)</code>	Animation	Cria uma animação
<code>getAnimations()</code>	Animation[]	Lista as animações ativas
<code>addEventListener(tipo, cb, opts)</code>	void	Adiciona um listener de evento
<code>removeEventListener(tipo, cb)</code>	void	Remove um listener de evento
<code>dispatchEvent(evento)</code>	Boolean	Dispara um evento

Glossário de Termos

Termo	Definição
DOM	Document Object Model — representação em árvore de um documento HTML/XML
Nó (Node)	Unidade básica da árvore do DOM
Elemento	Nó do tipo Element, representando uma tag HTML
Seletor	Padrão CSS usado para encontrar elementos no DOM
Evento	Sinal de que algo aconteceu no sistema
Listener	Função registrada para responder a um evento
Bubbling	Propagação de um evento do elemento alvo para seus ancestrais
Captura	Propagação de um evento da raiz até o elemento alvo
Delegação	Técnica de usar um único listener no pai para eventos dos filhos
Reflow	Recálculo do layout pelo navegador
Repaint	Redesenho visual sem recálculo de layout
Viewport	Área visível da janela do navegador
IIFE	Immediately Invoked Function Expression — função executada imediatamente
Debounce	Técnica para atrasar a execução até que as chamadas parem
Throttle	Técnica para limitar a frequência de execução de uma função
XSS	Cross-Site Scripting — vulnerabilidade de injeção de scripts
ARIA	Accessible Rich Internet Applications — atributos de acessibilidade
BOM	Browser Object Model — APIs do navegador além do DOM
API	Application Programming Interface — interface de programação
Callback	Função passada como argumento para ser chamada posteriormente
Polyfill	Código que implementa funcionalidade ausente em navegadores antigos

Termo	Definição
Lazy Loading	Carregamento de recursos apenas quando necessário
Virtual DOM	Representação em memória do DOM usada por frameworks como React
Shadow DOM	DOM encapsulado usado em Web Components
Web Components	Conjunto de APIs para criar componentes HTML reutilizáveis

Fim da Apostila — JavaScript DOM: O Guia Definitivo

Manus AI — 2026 — Todos os direitos reservados

Capítulo 31: Dicas Avançadas e Truques Profissionais

31.1 Usando `Symbol` para Dados Privados em Componentes DOM

```

// Usando Symbol para armazenar dados privados em elementos DOM
const _dados = Symbol("dados");
const _estado = Symbol("estado");

class ComponenteContador {
  constructor(elemento) {
    this.elemento = elemento;
    this.elemento[_dados] = { valor: 0, passo: 1 };
    this.elemento[_estado] = { animando: false };
    this._renderizar();
    this._vincularEventos();
  }

  _renderizar() {
    const dados = this.elemento[_dados];
    this.elemento.innerHTML = `
      <div class="contador-display">${dados.valor}</div>
      <div class="contador-controles">
        <button class="btn-decrementar">-</button>
        <input type="number" class="passo" value="${dados.passo}" min="1">
        <button class="btn-incrementar">+</button>
      </div>
    `;
  }

  _vincularEventos() {
    this.elemento.addEventListener("click", (e) => {
      const dados = this.elemento[_dados];
      if (e.target.classList.contains("btn-incrementar")) {
        dados.valor += dados.passo;
        this._atualizar();
      }
      if (e.target.classList.contains("btn-decrementar")) {
        dados.valor -= dados.passo;
        this._atualizar();
      }
    });
  }
}

```

```
        this.elemento.querySelector(".passo").addEventListener("change", (e) => {
            this.elemento[_dados].passo = parseInt(e.target.value) || 1;
        });
    }

    _atualizar() {
        const display = this.elemento.querySelector(".contador-display");
        display.textContent = this.elemento[_dados].valor;
        display.classList.add("atualizado");
        display.addEventListener("animationend", () => display.classList.remove("atualizado"), { once: true });
    }
}

// Inicializando componentes
document.querySelectorAll(".componente-contador").forEach(el => {
    new ComponenteContador(el);
});
```

31.2 Criando um Sistema de Roteamento SPA Simples

```

const rotas = {};
const rotaAtual = { caminho: null, componente: null };

function registrarRota(caminho, renderizador) {
  rotas[caminho] = renderizador;
}

function navegarPara(caminho, estado = {}) {
  if (rotaAtual.caminho === caminho) return;

  const renderizador = rotas[caminho] || rotas["*"] || (() => "<h1>404 – Página não encontrada</h1>");

  history.pushState({ caminho, ...estado }, "", caminho);
  rotaAtual.caminho = caminho;

  const app = document.getElementById("app");

  // Transição de saída
  app.style.opacity = "0";
  app.style.transition = "opacity 0.2s";

  setTimeout(() => {
    app.innerHTML = renderizador(estado);
    app.style.opacity = "1";

    // Dispara evento customizado
    document.dispatchEvent(new CustomEvent("rota:mudou", { detail: { caminho } }));
  }, 200);
}

// Interceptar cliques em links internos
document.addEventListener("click", function(e) {
  const link = e.target.closest("a[href^='/']");
  if (link && !link.hasAttribute("target")) {
    e.preventDefault();
    navegarPara(link.getAttribute("href"));
  }
});

```

```
// Lidar com navegação pelo histórico (botão voltar/avançar)
window.addEventListener("popstate", function(e) {
  if (e.state?.caminho) {
    navegarPara(e.state.caminho, e.state);
  }
});

// Registrando rotas
registrarRota("/", () => `

# 


```

31.3 Implementando um Sistema de Temas (Dark/Light Mode)

```

const CHAVE_TEMA = "preferencia-tema";

const gerenciadorTema = {
  // Obtém o tema preferido (salvo ou do sistema)
  obterTema() {
    const salvo = localStorage.getItem(CHAVE_TEMA);
    if (salvo) return salvo;

    // Detecta a preferência do sistema operacional
    return window.matchMedia("(prefers-color-scheme: dark)").matches
      ? "escuro"
      : "claro";
  },

  // Aplica o tema ao documento
  aplicar(tema) {
    document.documentElement.setAttribute("data-tema", tema);
    document.documentElement.style.setProperty(
      "--cor-fundo",
      tema === "escuro" ? "#1a1a2e" : "#ffffff"
    );
    document.documentElement.style.setProperty(
      "--cor-texto",
      tema === "escuro" ? "#e8f0fe" : "#1a1a2e"
    );

    // Atualiza o ícone do botão de alternância
    const btnTema = document.querySelector("#btn-tema");
    if (btnTema) {
      btnTema.textContent = tema === "escuro" ? "☀️ Modo Claro" : "🌙 Modo Escuro";
      btnTema.setAttribute("aria-label", `Ativar modo ${tema === "escuro" ? "claro" : "escuro"}`);
    }

    localStorage.setItem(CHAVE_TEMA, tema);

    document.dispatchEvent(new CustomEvent("tema:mudou", { detail: { tema } }));
  },
};

```

```
// Alterna entre os temas
alternar() {
    const temaAtual = this.obterTema();
    this.aplicar(temaAtual === "escuro" ? "claro" : "escuro");
},

// Inicializa o gerenciador
inicializar() {
    this.aplicar(this.obterTema());

    // Escuta mudanças na preferência do sistema
    window.matchMedia("(prefers-color-scheme: dark)").addEventListener("change", (e) => {
        if (!localStorage.getItem(CHAVE_TEMA)) {
            this.aplicar(e.matches ? "escuro" : "claro");
        }
    });

    // Vincula o botão de alternância
    document.querySelector("#btn-tema")?.addEventListener("click", () => this.alternar());
}

};

gerenciadorTema.inicializar();
```

31.4 Gerenciando Foco e Acessibilidade em Componentes Complexos

```
// Implementação de um componente Accordion acessível
class Accordion {
  constructor(container) {
    this.container = container;
    this.itens = container.querySelectorAll(".accordion-item");
    this._inicializar();
  }

  _inicializar() {
    this.itens.forEach((item, indice) => {
      const cabecalho = item.querySelector(".accordion-header");
      const conteudo = item.querySelector(".accordion-content");
      const id = `accordion-item-${indice}`;

      // Configurar atributos ARIA
      cabecalho.setAttribute("id", `${id}-header`);
      cabecalho.setAttribute("aria-controls", `${id}-content`);
      cabecalho.setAttribute("aria-expanded", "false");
      cabecalho.setAttribute("role", "button");
      cabecalho.setAttribute("tabindex", "0");

      conteudo.setAttribute("id", `${id}-content`);
      conteudo.setAttribute("role", "region");
      conteudo.setAttribute("aria-labelledby", `${id}-header`);
      conteudo.setAttribute("hidden", "");

      // Eventos de clique e teclado
      cabecalho.addEventListener("click", () => this._alternar(item));
      cabecalho.addEventListener("keydown", (e) => {
        if (e.key === "Enter" || e.key === " ") {
          e.preventDefault();
          this._alternar(item);
        }
        if (e.key === "ArrowDown") {
          e.preventDefault();
          this._focarProximo(indice);
        }
        if (e.key === "ArrowUp") {

```



```

        e.preventDefault();
        this._focarAnterior(indice);
    }
    if (e.key === "Home") {
        e.preventDefault();
        this._focarPrimeiro();
    }
    if (e.key === "End") {
        e.preventDefault();
        this._focarUltimo();
    }
    });
});
}

_alternar(item) {
    const cabecalho = item.querySelector(".accordion-header");
    const conteudo = item.querySelector(".accordion-content");
    const estaAberto = cabecalho.getAttribute("aria-expanded") === "true";

    if (estaAberto) {
        cabecalho.setAttribute("aria-expanded", "false");
        conteudo.setAttribute("hidden", "");
    } else {
        cabecalho.setAttribute("aria-expanded", "true");
        conteudo.removeAttribute("hidden");
    }
}

_focarProximo(indiceAtual) {
    const proximo = this.itens[indiceAtual + 1];
    if (proximo) proximo.querySelector(".accordion-header").focus();
}

_focarAnterior(indiceAtual) {
    const anterior = this.itens[indiceAtual - 1];
    if (anterior) anterior.querySelector(".accordion-header").focus();
}

```

```
_focarPrimeiro() {  
    this.itens[0].querySelector(".accordion-header").focus();  
}  
  
_focarUltimo() {  
    this.itens[this.itens.length - 1].querySelector(".accordion-header").focus();  
}  
}  
  
// Inicializando todos os accordions da página  
document.querySelectorAll(".accordion").forEach(el => new Accordion(el));
```

Considerações Finais

O Caminho do Desenvolvedor Frontend

Ao concluir esta apostila, você adquiriu um conhecimento sólido e abrangente sobre o Document Object Model do JavaScript. Você aprendeu desde os conceitos mais fundamentais — como a estrutura da árvore do DOM e os tipos de nós — até técnicas avançadas como o `MutationObserver`, a Web Animations API, e a criação de componentes acessíveis.

O DOM é a fundação sobre a qual todo o desenvolvimento Frontend moderno é construído. Frameworks como React, Vue e Angular são, em essência, abstrações sofisticadas que gerenciam o DOM por você. Ao compreender profundamente como o DOM funciona, você se torna capaz de:

- Depurar problemas que as abstrações dos frameworks escondem.
- Tomar decisões de performance informadas.
- Criar bibliotecas e utilitários reutilizáveis.
- Entender o que acontece "por baixo dos panos" em qualquer aplicação web.

Próximos Passos Recomendados

Após dominar o DOM, os próximos tópicos naturais para aprofundamento são:

Tópico	Por que Aprender
Fetch API e Promises	Comunicação assíncrona com servidores
ES6+ Moderno	Módulos, classes, destructuring, async/await
Web Components	Componentes nativos reutilizáveis com Shadow DOM
Service Workers	PWAs, cache offline, notificações push
WebSockets	Comunicação em tempo real bidirecional
Canvas API Avançada	Jogos, visualizações de dados, edição de imagens
WebGL	Gráficos 3D no navegador
React / Vue / Angular	Frameworks para aplicações complexas
TypeScript	JavaScript com tipagem estática
Testes Automatizados	Jest, Testing Library, Cypress

Referências e Recursos Adicionais

Para continuar aprendendo e manter-se atualizado, os seguintes recursos são altamente recomendados:

Recurso	URL	Descrição
MDN Web Docs	developer.mozilla.org	Documentação oficial e completa
WHATWG DOM Spec	dom.spec.whatwg.org	Especificação viva do DOM
Can I Use	caniuse.com	Compatibilidade de recursos entre navegadores
JavaScript.info	javascript.info	Tutorial moderno e aprofundado
W3C	w3.org	Padrões e especificações web
Web.dev	web.dev	Boas práticas do Google

Esta apostila foi produzida com dedicação para ser o guia mais completo sobre JavaScript DOM em língua portuguesa. Esperamos que ela sirva como uma referência constante em sua jornada como desenvolvedor web.

Manus AI — 2026